



Fachhochschul-Bachelorstudiengang
SOFTWARE ENGINEERING
A-4232 Hagenberg, Austria

Digitale Signalverarbeitung mit FAUST

Bachelorarbeit

zur Erlangung des akademischen Grades
Bachelor of Science in Engineering

Eingereicht von

Xianglei Wu

Begutachtet von FH-Prof. DI (FH) Dr. Erik Pitzer

Hagenberg, 05.08 2026

Erklärung

Ich erkläre eidesstattlich, dass ich die vorliegende Arbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen nicht benutzt und die den benutzten Quellen entnommenen Stellen als solche gekennzeichnet habe. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt.

Die vorliegende, gedruckte Bachelorarbeit ist identisch zu dem elektronisch übermittelten Textdokument.

05.08.2026
Datum

Unterschrift

Inhaltsverzeichnis

Abstract	vi
Kurzfassung	vii
1 Einleitung	1
1.1 Forschungsfrage und Zielsetzung	2
1.2 Aufbau der Arbeit	3
2 Grundlagen	4
2.1 Digitale Audiosignale und blockweise Verarbeitung	4
2.1.1 Abtastung und diskrete Sample-Folgen	4
2.1.2 Audio-Blöcke und Echtzeitdeadline	5
2.2 Elementare DSP-Bausteine	6
2.2.1 Verstärkungsstufe	6
2.2.2 Rekursionsfilter (IIR-Filter) und Biquad-Struktur	7
2.2.3 FIR-Filter und Faltung	8
2.3 Frequenzanalyse mit diskreter Fourier-Transformation (DFT) und Fast-Fourier-Transformation (FFT)	9
2.3.1 Zeitbereich, Frequenzbereich und DFT	9
2.3.2 Fast-Fourier-Transformation	9
2.4 Performanzrelevante Implementierungsaspekte	10
2.4.1 Compileroptimierung und automatische Vektorisierung	10
2.4.2 Messgrößen	10
2.5 Zusammenfassung	11
3 Faust: Functional Audio Stream	12
3.1 Einführung	12
3.2 Programmmodell	13
3.3 Syntax: Blockdiagramm-Komposition	13
3.4 Semantik	14
3.5 Compiler-Architektur	15
3.6 Codegenerierung	16
3.6.1 Skalarer Modus	16
3.6.2 Vektor-Modus	17
3.6.3 Parallel-Modus	17
3.6.4 Generierte Code	17

3.7	Backends und Architekturdateien	17
3.8	Performance	18
3.9	Zusammenfassung	18
4	Versuchsdesign und Benchmark-Methodik	19
4.1	Definition der untersuchten DSP-Bausteine	19
4.1.1	Fast-Fourier-Transformation	25
4.2	Benchmark-Tools und Messgrößen	26
4.2.1	Messwerkzeuge	26
4.2.2	Messgrößen	27
4.3	Hardwareumgebung	28
4.4	Compilerkonfigurationen	29
4.4.1	Aufruf des Faust-Compilers	29
4.4.2	Flag-Matrix	29
4.5	Testdatengenerierung	30
4.6	Auswertungsverfahren	31
4.6.1	Kennzahlen und statistische Prüfung	31
4.6.2	Auflösungsgrenze des Messaufbaus	31
4.7	Übersicht der Messreihen	32
4.7.1	Voraussetzungen	32
4.7.2	Laufzeit und Compilerverhalten	33
4.7.3	Ressourcenverbrauch	33
4.7.4	Entwicklungsaufwand	34
4.7.5	Einordnung und Grenzen des Messplans	34
4.8	Zusammenfassung	35
5	Implementierung der Benchmarks	36
5.1	Umsetzung des FAUST- und C++-Code	37
5.2	Gemeinsame Test-Schnittstelle	39
5.3	Build-System	40
5.4	Sicherstellung einer fairen Vergleichsbasis	41
5.5	Verifikation der Ergebnisse	42
6	Evaluation der Messergebnisse	44
6.1	Mikrobenchmark-Ergebnisse	44
6.2	Einfluss von Compileroptimierungen und Codegenerierung	44
6.3	Numerische Genauigkeit	44
6.4	Codelänge und Implementierungskomplexität	44
6.5	Zusammenfassung der Messergebnisse	44
7	Diskussion	45
7.1	Interpretation der Ergebnisse im Hinblick auf die Forschungsfrage	45
7.2	Limitationen des Versuchsaufbaus	45
7.3	Bedrohung der Validität	45
8	Zusammenfassung und Ausblick	46
8.1	Zusammenfassung der zentralen Ergebnisse	46

Inhaltsverzeichnis	v
8.2 Ausblick auf künftige Forschungsarbeiten	46
Quellenverzeichnis	47
Literatur	47
Online-Quellen	48

Abstract

This should be a 1-page (maximum) summary of your work in English.

Kurzfassung

An dieser Stelle steht eine Zusammenfassung der Arbeit, Umfang max. 1 Seite. ...

Kapitel 1

Einleitung

Die digitale Signalverarbeitung (Digital Signal Processing, DSP) ist ein wichtiges Anwendungsfeld in der Informatik. Besonderen Stellenwert hat sie Sie findet unter anderem Anwendung in der Telekommunikation, Audiotechnik, Sprachverarbeitung und Medizintechnik (Ifeachor & Jervis, 2002). In diesen Anwendungsbereichen sind Laufzeiteffizienz, deterministische Latenz und numerische Präzision von besonderer Bedeutung.

DSP-Algorithmen werden traditionell in C oder C++ implementiert, unter anderem wegen des direkten Hardwarezugriffs und der stark optimierten Codeerzeugung (Levine & Skolnick, 1997).

Die manuelle Übertragung mathematischer Modelle in ausführbaren Code ist jedoch mit erheblichem Aufwand verbunden und erschwert die Portierung auf unterschiedliche Zielplattformen. In den letzten zwanzig Jahren wurden daher domänenspezifische Sprachen (Domain-Specific Languages, DSLs) für die Audio-Signalverarbeitung entwickelt, die durch Abstraktion und Code-Generierung diese Herausforderungen adressieren sollen.

parentcite[S. 2-6]orlarey:hal-02159014

Orlarey et al. (2009) beschreibt die domänenspezifische Sprache FAUST (Functional Audio Stream), die am GRAME in Lyon entwickelt wurde, als eine funktionale, blockbasierte Programmiersprache. Sie wurde speziell für Audio-DSP entwickelt. Programme bestehen aus einer algebraischen Kombination von Signalverarbeitungsoperatoren. Die aktuelle FAUST-Dokumentation beschreibt die Codegenerierung für mehrere Zielsprachen und Zielplattformen, darunter C++, Rust, JavaScript und WebAssembly (GRAME - Centre National de Création Musicale, 2025). Sie dokumentiert zudem Architekturdateien, mit denen aus einer FAUST-Quellbeschreibung unter anderem Audio-Plug-ins, Anwendungen für eingebettete Systeme und Webanwendungen erzeugt werden können (GRAME - Centre National de Création Musicale, 2025).

Die praktische Leistungsfähigkeit dieses Ansatzes wird am Beispiel des FAUST-STK untersucht. FAUST-STK ist eine Sammlung virtueller Musikinstrumente, deren Modelle überwiegend auf Wellenleitersynthese und teilweise auf modaler Synthese beruhen (Michon & Smith, 2011, S. 1). Die aus dem Synthesis ToolKit und SynthBuilder übernommenen Modelle wurden für die FAUST-Semantik angepasst und hinsichtlich ihrer Effizienz optimiert (Michon & Smith, 2011, S. 5–6).

Die Quelltextgrößenanalyse zeigt, dass die FAUST-Implementierungen bei den un-

tersuchten Modellen kompakter sind als die entsprechenden STK-C++- Implementierungen. Der ausgewiesene Größenvorteil beträgt je nach Modell zwischen 22,91 Prozent und 80,20 Prozent (Michon & Smith, 2011, Tab. 1, S. 6). Zusätzlich vergleicht die Studie die CPU-Last von Pure-Data-Plug-ins, die auf STK-C++-Code beziehungsweise FAUST-generiertem Code basieren (Michon & Smith, 2011, Tab. 2, S. 6).

Für die praktische Eignung ist es erforderlich, dass der durch FAUST generierte Code sowohl funktional korrekt als auch laufzeiteffizient ist. Scaringella et al. (2003) thematisieren die automatische Vektorisierung, während Letz et al. (2010) Möglichkeiten der Parallelisierung im FAUST-Umfeld behandeln. Die FAUST-Dokumentation verweist zudem auf Optimierungen, die den vollständigen Signalfluss berücksichtigen (Orlarey et al., 2009). Diese Eigenschaften lassen jedoch keine allgemeine Aussage darüber zu, ob FAUST-generierter C++-Code einer manuell implementierten C++-Referenz bei konkreten DSP-Bausteinen überlegen, gleichwertig oder unterlegen ist.

Direkte Vergleiche liegen für zeitbereichsbasierte Algorithmen vor. Für den Reverb-Algorithmus *Freeverb* berichten Orlarey et al. (2009) Benchmarkergebnisse, deren Ausprägung von Compiler und Codegenerierungsstrategie abhängt. Die in der vorliegenden Arbeit berücksichtigte Vergleichsliteratur untersucht überwiegend zeitbereichsbasierte Algorithmen wie Reverbs, Delays und physikalische Modelle. Ergänzend wird daher ein Vergleich FFT-bezogener Bausteine durchgeführt. Für einen fairen Vergleich folgen beide Implementierungen demselben algorithmischen Ablauf und werden über eine gemeinsame Schnittstelle in die Benchmark-Infrastruktur eingebunden. Auf explizit programmierte Single-Instruction-Multiple-Data-Intrinsics (SIMD-Intrinsics) wird verzichtet. Die automatische Vektorisierung des Compilers bleibt in beiden Varianten aktiv.

1.1 Forschungsfrage und Zielsetzung

Diese Arbeit untersucht das Verhalten von FAUST-generiertem C++-Code im Vergleich zu einer manuell implementierten C++-Referenz für grundlegende DSP-Bausteine. Berücksichtigt werden eine Gain-Stufe, ein Biquad-Filter, Finite-Impulse-Response-Filter (FIR-Filter) verschiedener Längen sowie FFTs unterschiedlicher Größen. Die Anforderungen umfassen reproduzierbare Vergleiche unter unterschiedlichen Datenabhängigkeiten und Skalierungseigenschaften. Neben der Laufzeit werden auch Speicherverbrauch und Binärgröße analysiert. Zusätzlich erfolgt eine Analyse der Codelänge und Implementierungskomplexität.

Die übergeordnete Forschungsfrage lautet:

Wie unterscheidet sich FAUST-generierter C++-Code von einer manuell implementierten C++-Referenz bei ausgewählten DSP-Bausteinen und FFTs hinsichtlich Laufzeit, Speicherverbrauch, Binärgröße, Codelänge und Implementierungskomplexität?

Zur Beantwortung dieser Forschungsfrage werden folgende Teilfragen untersucht:

- Wie groß ist der Performanzunterschied hinsichtlich der CPU-Zeit pro Audio-Block zwischen FAUST-generiertem und manuell implementiertem C++-Code bei Gain-Stufen, Biquad-Filtern, FIR-Filtern und FFTs?
- Wie verändert sich der Laufzeitunterschied zwischen beiden Implementierungen in Abhängigkeit von Blockgröße, FIR-Filterlänge und FFT-Größe?

- Wie unterscheiden sich FAUST-generierter und manuell implementierter C++-Code hinsichtlich des Speicherverbrauchs während der Ausführung der untersuchten DSP-Bausteine?
- Welche Unterschiede bestehen zwischen FAUST-generiertem und manuell implementiertem C++-Code hinsichtlich der Binärgröße der erzeugten Programme?
- Welchen Einfluss haben Compileroptimierungen und automatische Vektorisierung auf beide Implementierungen?
- Welche Unterschiede bestehen hinsichtlich Codelänge und Implementierungskomplexität der beiden Ansätze?

1.2 Aufbau der Arbeit

Die Bachelorarbeit gliedert sich in acht Kapitel. Sie behandelt die theoretischen Grundlagen, die Beschreibung von FAUST, das Versuchsdesign, die Implementierung der Benchmark-Kerne, die Evaluation und die Einordnung der Ergebnisse.

Nach dieser Einleitung vermittelt Kapitel 2 die für die Arbeit relevanten Grundlagen der digitalen Signalverarbeitung. Dazu gehören Audio-DSP, Verstärkungsstufen, rekursive Filter und FIR-Filter, die Fast-Fourier-Transformation sowie Anforderungen an Echtzeitverarbeitung.

Kapitel 3 stellt die domänenspezifische Sprache FAUST vor. Es erläutert die funktionale Syntax und Semantik, die zugrunde liegende Compiler-Architektur sowie die für die Arbeit relevanten Codegenerierungs-Backends.

Kapitel 4 beschreibt das Versuchsdesign und die Benchmark-Methodik. Es definiert die untersuchten DSP-Bausteine, die Messgrößen, die Zielplattformen, die Compilerkonfigurationen und die Maßnahmen zur Sicherung reproduzierbarer Messungen.

Kapitel 5 dokumentiert die praktische Umsetzung der FAUST- und C++-Varianten. Neben den DSP-Bausteinen werden die gemeinsame Schnittstelle, das Build-System und die Maßnahmen zur Gewährleistung einer fairen Vergleichsbasis beschrieben.

Kapitel 6 enthält die Evaluation. Die Ergebnisse der Mikrobenchmarks werden hinsichtlich Laufzeit, Speicherverbrauch und Binärgröße ausgewertet. Zusätzlich werden Skalierungseffekte durch unterschiedliche Blockgrößen, Filterlängen und FFT-Größen sowie der Einfluss von Compileroptimierungen analysiert. Im Anschluss wird die Erfahrung mit der Handhabung von FAUST bewertet.

Kapitel 7 diskutiert die Ergebnisse im Hinblick auf die Forschungsfrage. Dabei werden Limitationen des Versuchsaufbaus, Bedrohungen der Validität und sowie die Codelänge und Implementierungskomplexität analysiert.

Kapitel 8 fasst die zentralen Ergebnisse zusammen und gibt einen Ausblick auf mögliche Erweiterungen, etwa die Untersuchung vollständiger DSP-Pipelines oder weiterer Zielplattformen.

Kapitel 2

Grundlagen

Dieses Kapitel stellt die signaltheoretischen und laufzeitbezogenen Grundlagen vor. Sie sind nötig, um Faust-generierten mit manuell geschriebenem C++-Code zu vergleichen. Der Fokus liegt auf digitalen Audiosignalen, einer Verstärkungsstufe (Gain-Stufe), Biquad- und Finite-Impulse-Response-(FIR)-Filtern, der Fast-Fourier-Transformation (FFT) und den Anforderungen der blockbasierten Echtzeitverarbeitung.

2.1 Digitale Audiosignale und blockweise Verarbeitung

2.1.1 Abtastung und diskrete Sample-Folgen

Ein analoges Audiosignal ist zeitkontinuierlich. Für die digitale Verarbeitung wird es in regelmäßigen Zeitabständen abgetastet. Es wird als diskrete Folge von Sample-Werten dargestellt (Rossing, 2007). Jedes Sample hat einen festen zeitlichen Abstand zum vorherigen Sample (Rossing, 2007). Die Abtastfrequenz f_s bestimmt die Anzahl der Messungen pro Sekunde. Das Abtastintervall beträgt

$$T_s = \frac{1}{f_s}. \quad (2.1)$$

Die Beziehung zwischen Abtastfrequenz und Abtastintervall ergibt sich aus der Definition der Abtastfrequenz für digitale Audiosignale (Rossing, 2007, S. 520-522). Die digitale Darstellung eines kontinuierlichen Signals $x(t)$ lautet damit

$$x[n] = x(nT_s), \quad n \in \mathbb{Z}. \quad (2.2)$$

Diese zeitdiskrete Darstellung beschreibt die Umwandlung eines kontinuierlichen Signals in eine Folge von Abtastwerten, siehe (Rossing, 2007, S. 520–522). Das Nyquist-Shannon-Kriterium fordert, dass die Abtastfrequenz mindestens doppelt so hoch wie die höchste im Signal enthaltene Frequenz sein muss. Andernfalls können Frequenzanteile nicht eindeutig rekonstruiert werden (Tan & Jiang, 2018, Kap. 2.1, S. 19–26). Die Abtastung mit einer Rate oberhalb von $2f_B$ entspricht die Nyquist-Abtastung und behandelt die zugehörige Bandbegrenzung als Voraussetzung einer aliasfreien digitalen Darstellung (Zölzer, 2022, S. 63–65).

$$f_s \geq 2f_{\max}. \quad (2.3)$$

In der vorliegenden Arbeit werden Audiosignale als Folgen von Gleitkommawerten verarbeitet. Die Quantisierung ist daher nur als Übergang von der analogen in die digitale Darstellung relevant. Eine separate Untersuchung der Bittiefe oder der Quantisierungsqualität ist nicht Gegenstand der Benchmarks.

2.1.2 Audio-Blöcke und Echtzeitdeadline

Audiosysteme zur digitalen Signalverarbeitung (DSP) können Eingangssamples entweder samplebasiert oder in Blöcken verarbeiten, siehe (Tan & Jiang, 2018, S. 727). Blockbasierte Verfahren kommen vor allem bei der digitalen Filterung, Faltung, FFT und anderen echtzeitfähigen DSP-Anwendungen eingesetzt (Tan & Jiang, 2018, S. 727). Ein Block besteht aus N aufeinanderfolgenden Samples. Bei einer Abtastfrequenz f_s und einer Abtastperiode $T_s = 1/f_s$ besitzt ein Block die zeitliche Dauer

$$T_{\text{Block}} = NT_s = \frac{N}{f_s}. \quad (2.4)$$

Die Blockdauer hängt von der Anzahl der Samples und dem zeitlichen Abstand zwischen zwei aufeinanderfolgenden Samples ab. Bei der überlappenden Blockverarbeitung werden die Eingangsdaten in zeitlich versetzten Analysefenstern verarbeitet, wobei aufeinanderfolgende Blöcke mit N Samples einen gemeinsamen Bereich haben (Tan & Jiang, 2018, S. 499). Bei einer Überlappung von 50 Prozent besteht der zweite Block aus der zweiten Hälfte des ersten Blocks und einer neuen zweiten Hälfte (Tan & Jiang, 2018, S. 499). Die Rekonstruktion der überlappenden Ausgangsblöcke erfolgt anschließend mittels Overlap-Add (Tan & Jiang, 2018, S. 500). Für eine Echtzeitverarbeitung muss die Berechnung rechtzeitig vor dem Eintreffen des nächsten Datenabschnitts beziehungsweise dessen Ausgabe abgeschlossen sein. Für eine samplebasierte Echtzeitverarbeitung müssen die Analog-Digital-Umwandlung (ADC), die DSP-Verarbeitung und die Ausgabe innerhalb des zeitlichen Intervalls bis zum nächsten Ausgabezeitpunkt abgeschlossen werden (Tan & Jiang, 2018, S. 761). Für eine blockbasierte Verarbeitung lässt sich diese Echtzeitbedingung durch

$$T_{\text{exec}} \leq T_{\text{Deadline}} \quad (2.5)$$

ausdrücken. Dabei steht T_{exec} für die tatsächliche Ausführungszeit des Algorithmus. Die Deadline hängt davon ab, wie die Blockverarbeitung organisiert ist. Bei nicht überlappenden Blöcken entspricht die verfügbare Zeit der Blockdauer:

$$T_{\text{Deadline}} = T_{\text{Block}} = \frac{N}{f_s}. \quad (2.6)$$

Bei überlappender Verarbeitung startet für alle H Samples ein neuer Verarbeitungszyklus. Die Zeitspanne zwischen zwei aufeinanderfolgenden Zyklen entspricht deshalb der Hop-Zeit.

$$T_{\text{Deadline}} = T_{\text{Hop}} = \frac{H}{f_s}. \quad (2.7)$$

Die Hop-Zeit ist also die wichtigste Rechenzeit für eine überlappende, blockbasierte Verarbeitung in Echtzeit. Die Blockgröße und die Hop-Größe wirken sich jeweils auf verschiedene Eigenschaften des Systems aus. Eine kleinere Blockgröße senkt die Latenz, die durch das Puffern entsteht. Gleichzeitig verringert sie jedoch die verfügbare Zeit für

die Verarbeitung (Tan & Jiang, 2018, S. 499, 500, 761). Bei einer Überlappung von 50 Prozent gilt zum Beispiel

$$H = \frac{N}{2}. \quad (2.8)$$

Die Blockgröße und die Hop-Größe sind deshalb zentrale Parameter für die Konfiguration und Bewertung von Echtzeit-Benchmarks (Zölzer, 2022).

2.2 Elementare DSP-Bausteine

2.2.1 Verstärkungsstufe

Eine Verstärkungsstufe multipliziert jedes Eingangssignal mit einem konstanten Verstärkungsfaktor g . Für ein Eingangssignal $x[n]$ ergibt sich das Ausgangssignal $y[n]$ zu

$$y[n] = g \cdot x[n]. \quad (2.9)$$

Diese Skalierung ist eine grundlegende lineare Operation in der digitalen Audioverarbeitung (Smith, 2007). Der Baustein benötigt pro Sample eine Multiplikation und dient daher als einfacher Referenzfall zum Vergleich des von Faust erzeugten Codes mit der C++-Referenz. Bei einer Angabe der Verstärkung in Dezibel gilt

$$g = 10^{G_{\text{dB}}/20}. \quad (2.10)$$

Diese Umrechnung ergibt sich aus der Definition eines Amplitudenverhältnisses in Dezibel (Smith, 2007). Abbildung 2.1 zeigt eine Verstärkungsstufe mit $g = 0.5$. Das Ausgangssignal besitzt für jedes Sample die halbe Amplitude des Eingangssignals. Das entspricht einer Absenkung von ungefähr -6.02 dB. Die Verstärkungsstufe bildet einen ein-

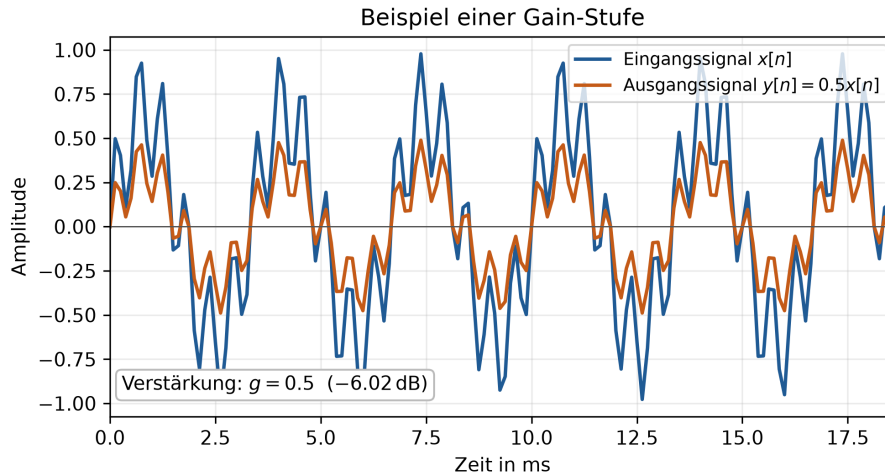


Abbildung 2.1: Beispiel einer Verstärkungsstufe mit $g = 0.5$. Die Amplitude jedes Eingangssamples wird halbiert. Eigene Darstellung nach (Smith, 2007).

fachen, vollständig datenparallelen Benchmark-Kern. Vektorisierung bezeichnet dabei

die Ausführung derselben Operation auf mehreren Datenwerten durch Single-Instruction-Multiple-Data-Instruktionen (SIMD-Instruktionen) (Scaringella et al., 2003). Eine sampleweise Skalierung weist daher ein hohes Vektorisierungspotenzial auf. Der Faust-Compiler kann solche Ausdrücke über seine Codegenerierungsstrategien in vektorisierbaren C++-Code übersetzen (Orlarey et al., 2009, S. 2–3, 15).

2.2.2 Rekursionsfilter (IIR-Filter) und Biquad-Struktur

Ein Rekursionsfilter, auch Infinite-Impulse-Response-Filter (IIR-Filter) genannt, benutzt neben aktuellen und vergangenen Eingangswerten auch vergangene Ausgangswerte. Die Rückkopplung führt dazu, dass der Filterzustand über mehrere Samples bestehen bleibt. Für die Untersuchung wird ein Biquad-Filter, also ein IIR-Filter zweiter Ordnung (Zölzer, 2022). Seine allgemeine Differenzengleichung lautet bei normiertem $a_0 = 1$

$$y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] - a_1y[n-1] - a_2y[n-2]. \quad (2.11)$$

IIR-Filter lassen sich durch solche Differenzengleichungen beschreiben. Die Koeffizienten b_i bilden den Vorwärtzweig. Die Koeffizienten a_i bestimmen den Rückkopplungsweig (Smith, 2007). Abbildung 2.2 zeigt die Wirkung eines Biquad-IIR-Tiefpasses zweiter Ordnung mit einer Grenzfrequenz von 800 Hz. Wie beim FIR-Beispiel enthält das synthetische Eingangssignal Anteile bei 300 Hz und 1800 Hz. Der Frequenzgang und das Ausgangssignal veranschaulichen die Dämpfung hoher Frequenzanteile durch die rekursive Filterstruktur. Eine IIR-Implementierung muss die vergangenen Ein- und Ausgangs-

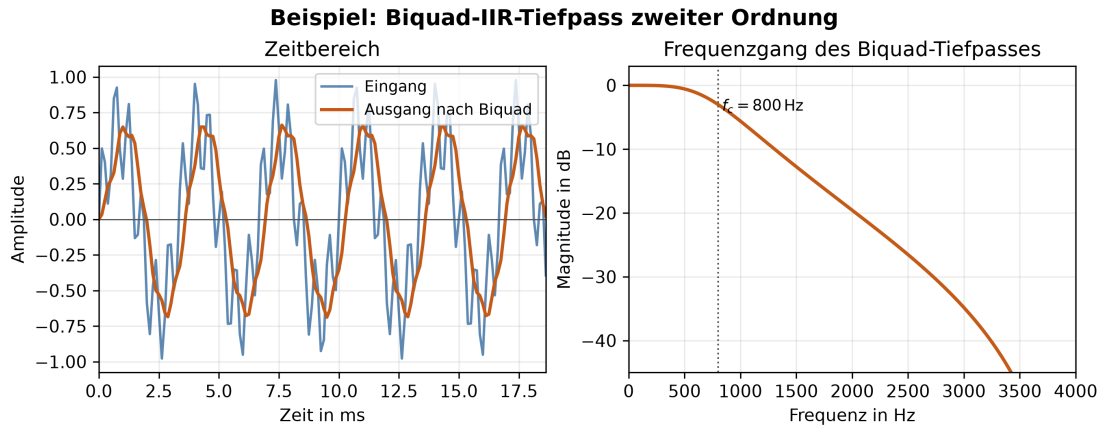


Abbildung 2.2: Beispiel eines Biquad-IIR-Tiefpasses zweiter Ordnung mit einer Grenzfrequenz von $f_c = 800$ Hz. Der Frequenzgang zeigt die Dämpfung hoher Frequenzanteile. Darstellung nach (Smith, 2007).

werte als Zustände speichern und bei jedem Sample aktualisieren. Für den Benchmark ist besonders wichtig, dass diese Rückkopplung Datenabhängigkeiten schafft und die Möglichkeit zur Vektorisierung im Vergleich zu nur vorwärts gerichteten Operationen einschränken kann. Deshalb unterscheidet man zwischen vektorisierbaren und skalaren Ausdrücken. Rekursive Filter sind dabei ein wichtiger Sonderfall (Scaringella et al., 2003).

2.2.3 FIR-Filter und Faltung

Ein Finite-Impulse-Response-Filter (FIR-Filter) hat keine Rückkopplung. Jedes Ausgangssample wird aus einer gewichteten Summe des aktuellen und der vorherigen Eingangssamples berechnet. Für eine Filterlänge L gilt

$$y[n] = \sum_{k=0}^{L-1} b_k x[n-k]. \quad (2.12)$$

Diese Differenzengleichung entspricht der direkten Faltungsimplementierung eines FIR-Filters (Smith, 2007). Abbildung 2.3 zeigt einen FIR-Tiefpass mit 31 Filterkoeffizienten (Taps) und einer Grenzfrequenz von 800 Hz. Das synthetische Eingangssignal enthält einen niederfrequenten Anteil bei 300 Hz. Es enthält außerdem einen höherfrequenten Anteil bei 1800 Hz. Der FIR-Filter dämpft den höherfrequenten Anteil. Im Unterschied

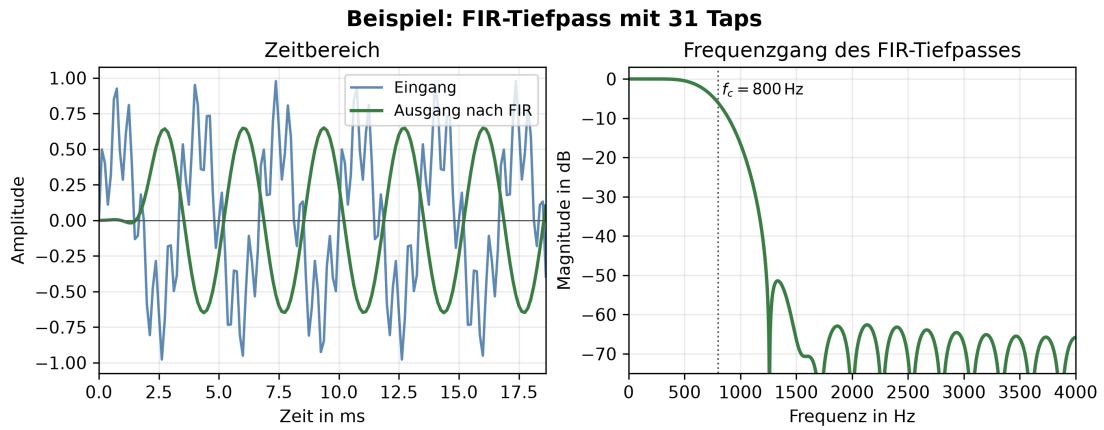


Abbildung 2.3: Beispiel eines FIR-Tiefpasses mit 31 Filterkoeffizienten und einer Grenzfrequenz von $f_c = 800$ Hz. Der Filter arbeitet ohne Rückkopplung und dämpft hohe Frequenzanteile. Darstellung nach (Smith, 2007).

zum Biquad hängt $y[n]$ nicht von früheren Ausgangswerten ab. Der Aufwand steigt mit der Zahl der Taps. Für jedes Ausgangssample liegt die asymptotische Komplexität daher

$$O(L). \quad (2.13)$$

Die lineare Abhängigkeit von der Filterlänge ergibt sich aus der direkten Auswertung der Faltungssumme (Smith, 2007). Die Filterlänge ist für die Evaluation wesentlich. Sie erhöht sowohl die Zahl der Rechenoperationen als auch die Größe des benötigten Zustands. Nichtrekursive Filterstrukturen können vollständig vektorisiert werden, sofern keine weiteren Datenabhängigkeiten entgegenstehen (Scaringella et al., 2003).

2.3 Frequenzanalyse mit diskreter Fourier-Transformation (DFT) und Fast-Fourier-Transformation (FFT)

2.3.1 Zeitbereich, Frequenzbereich und DFT

Im Zeitbereich zeigt ein digitales Signal seinen Amplitudenverlauf über dem Sample-Index. Im Frequenzbereich wird derselbe Signalabschnitt mit seinen spektralen Komponenten dar (Tan & Jiang, 2018, Kap. 4.1, 4.2, S. 97, 103). Die diskrete Fourier-Transformation (DFT) weist einer endlichen Folge $x[n]$ der Länge N komplexe Koeffizienten $X[k]$ zu.

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi kn/N}, \quad k = 0, \dots, N-1. \quad (2.14)$$

Die Bedeutung der Indizes n , k und N folgt unmittelbar aus der DFT-Definition (Tan & Jiang, 2018, S. 97). Der Betrag eines Koeffizienten beschreibt die Stärke der jeweiligen Frequenzkomponente, während sein Argument die zugehörige Phase angibt. Der Abstand zwischen benachbarten Frequenz-Bins beträgt

$$\Delta f = \frac{f_s}{N}. \quad (2.15)$$

Abbildung 2.4 vergleicht Zeitbereich und Frequenzbereich anhand eines Einzeltons und eines Mischsignals. Diese Frequenzauflösung ergibt sich aus der Abtastfrequenz und der Transformationslänge (Tan & Jiang, 2018, S. 101, 103). Eine größere Transformationslänge N verbessert die Frequenzauflösung, erhöht jedoch den Rechenaufwand.

2.3.2 Fast-Fourier-Transformation

Die Fast-Fourier-Transformation (FFT) berechnet dieselben DFT-Koeffizienten, aber mit einer effizienteren Zerlegung des Problems. Während eine direkte DFT eine Komplexität von

$$O(N^2) \quad (2.16)$$

aufweist, reduziert eine radix-2-FFT den Aufwand auf

$$O(N \log_2 N). \quad (2.17)$$

Diese Beschleunigung entsteht, weil die N -Punkt-DFT in kleinere Teiltransformationen zerlegt wird. Die asymptotischen Laufzeiten ergeben sich als Folge dieser Zerlegung (Tan & Jiang, 2018, S. 127–134). Die FFT-Größe ist im Benchmark ein wichtiger Skalierungsparameter. Sie beeinflusst sowohl den theoretischen Rechenaufwand als auch die praktische Speicher- und Cache-Nutzung (Tan & Jiang, 2018, S. 127–134). Dieses Kapitel behandelt nur die Vorwärtstransformation, da die Forschungsfrage einzelne FFT-Bausteine vergleicht. Eine vollständige Short-Time-Fourier-Transform-(STFT)-Pipeline oder eine Overlap-Add-Pipeline gehört nicht zu diesem Grundlagenkapitel.

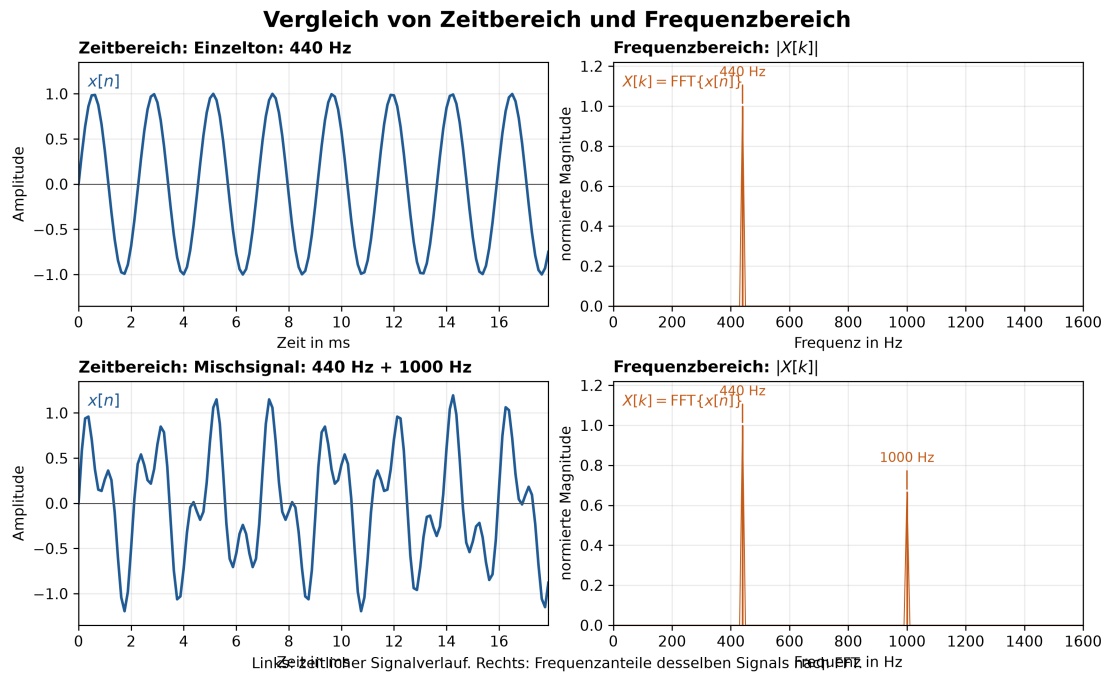


Abbildung 2.4: Vergleich von Zeitbereich und Frequenzbereich. Oben ist ein 440-Hz-Einzelton dargestellt, der im Spektrum einen dominanten Frequenzanteil bei 440 Hz aufweist. Unten ist ein Mischsignal aus 440 Hz und 1000 Hz dargestellt. Beide Anteile erscheinen im Frequenzbereich als separate Peaks. Darstellung basiert auf der DFT/FFT-Spektralanalyse (Tan & Jiang, 2018, S. 102).

2.4 Performanzrelevante Implementierungsaspekte

2.4.1 Compileroptimierung und automatische Vektorisierung

Die Laufzeit eines DSP-Kerns hängt nicht nur von seinem Algorithmus ab. Sie wird auch durch die Codegenerierung und die Optimierungen des Compilers beeinflusst. Bei automatischer Vektorisierung versucht der Compiler, mehrere unabhängige skalare Operationen mit SIMD-Instruktionen zusammenzufassen. Für Faust gibt es eine Analyse, die vektorisierbare Ausdrücke von solchen mit Datenabhängigkeiten trennt (Orlarey et al., 2009, S. 15). Diese Unterscheidung zeigt, warum Verstärkungsstufen und lange FIR-Strukturen andere Optimierungsmöglichkeiten haben können als rekursive Biquad-Filter. Die Arbeit vergleicht beide Implementierungen unter denselben Compiler- und Optimierungsbedingungen. Es kommen keine expliziten SIMD-Intrinsics zum Einsatz. So bleibt die Wirkung der automatischen Optimierung für Faust und C++ sichtbar. Die genaue Auswahl von Compiler, Flags und Zielplattform steht erst im Versuchsdesign.

2.4.2 Messgrößen

Die zentrale Messgröße ist die Prozessorzeit pro Audioblock. Ergänzend werden Speicherverbrauch und Binärgröße erfasst, da sie die praktische Einsetzbarkeit einer Implementierung neben der reinen Laufzeit beeinflussen. Codelänge und Implementierungs-

komplexität ergänzen den quantitativen Vergleich um Aspekte der Entwicklererfahrung. Die genaue Operationalisierung dieser Größen, die Zahl der Wiederholungen sowie die statistische Auswertung gehören in Kapitel 4.

2.5 Zusammenfassung

Die Grundlagen beschreiben die für die Benchmarks benötigte Verarbeitungskette von digitalen Samples über Gain-, IIR- und FIR-Operationen bis zur FFT. Blockgröße, Filterlänge und FFT-Größe bestimmen den Rechenaufwand und sind die zentralen Skalierungsparameter der Evaluation. Anschließend erläutern die folgenden Kapitel Faust und das konkrete Versuchsdesign.

Kapitel 3

Faust: Functional Audio Stream

3.1 Einführung

FAUST (Functional Audio Stream) ist eine domänenspezifische Programmiersprache für die Audio-Signalverarbeitung. Sie ist als effiziente Ergänzung zu C/C++ für Signalverarbeitungsbibliotheken, Audio-Plug-ins und eigenständige Anwendungen konzipiert (Orlarey et al., 2009).

Anders als visuelle Sprachen wie Max oder Pure Data ist FAUST eine kompilierte Sprache. Der FAUST-Compiler übersetzt FAUST-Programme direkt in C++-Code. Dieser Code ist lauffähig, selbstständig und benötigt keine externe Laufzeitumgebung. Er arbeitet deterministisch und mit konstantem Speicherverbrauch. Vgl. Orlarey et al., 2009, S. 2

Für die vorliegende Arbeit ist FAUST relevant, weil die Sprache Algorithmen der digitalen Signalverarbeitung (Digital Signal Processing, DSP) als funktionale Blockdiagramme beschreibt. Der Compiler erzeugt daraus C++-Code, der in dieselbe Benchmark-Infrastruktur wie eine manuell implementierte C++-Referenz eingebunden werden kann (Orlarey et al., 2009, S. 2–3). Der Vergleich betrifft die vom FAUST-Compiler abgeleitete und eine manuelle Implementierung desselben DSP-Algorithmus.

Die deklarative Blockdiagrammbeschreibung und die Trennung von Spezifikation und Implementierung können Codelänge und Implementierungskomplexität beeinflussen. Compileranalyse und Codegenerierung können sich zudem auf Laufzeit, Speicherverbrauch und Binärgröße auswirken. Ergebnisse aus FAUST-STK deuten für bestimmte physikalische Modelle auf kompaktere FAUST-Quelltexte hin (Michon & Smith, 2011, S. 5–6). Diese Ergebnisse sind nicht ohne Weiteres auf Gain-Stufen, Biquad- und FIR-Filter oder Fast-Fourier-Transformationen (FFTs) übertragbar. Die Überprüfung für diese DSP-Bausteine ist daher Gegenstand der vorliegenden Untersuchung.

Die Besonderheit von FAUST liegt in der formalen Semantik: Jedes Programm beschreibt eine mathematische Funktion, die Eingangssignale in Ausgangssignale transformiert. Der Compiler übersetzt nicht den Programmtext, sondern die dahinterstehende Funktion. Dadurch können zwei syntaktisch unterschiedliche, aber semantisch äquivalente Programme denselben Code erzeugen. (Orlarey et al., 2009)

Programm 3.1: Minimale FAUST-Signalprozessoren

```

1  process = 0;          // erzeugt Stille
2  process = _;          // kopiert Eingang auf Ausgang
3  process = +;          // addiert zwei Kanäle (Stereo zu Mono)
4

```

3.2 Programmmodell

FAUST kombiniert funktionale Programmierung mit Blockdiagrammen. Digitale Signale werden als diskrete Funktionen der Zeit modelliert. Signalprozessoren verarbeiten diese Signale, und Kompositionsoperatoren verbinden mehrere Signalprozessoren zu größeren Blockdiagrammen (Orlarey et al., 2009, S. 3–4).

Ein FAUST-Programm beschreibt keinen Klang, sondern einen Signalprozessor, also eine Maschine mit Eingängen und Ausgängen. Das Schlüsselwort **process** definiert diesen Signalprozessor. Programm 3.1 zeigt drei minimale Definitionen.

FAUST-Primitive funktionieren wie C-Funktionen, aber auf Signalen. Beispielsweise berechnet **sin** den Sinus jedes Samples eines Eingangssignals. Der Verzögerungsoperator **@** bildet ein Signal mit einer zeitvariablen Verzögerung ab (Orlarey et al., 2009).

3.3 Syntax: Blockdiagramm-Komposition

Tabelle 3.1 fasst die fünf Kompositionsoperatoren der FAUST-Blockdiagrammalgebra zusammen (Orlarey et al., 2009, S. 5).

Tabelle 3.1: Kompositionsoperatoren der Faust Block-Diagramm-Algebra

Operator	Bezeichnung	Beschreibung
$f \sim g$	Rekursive Komposition	Feedback-Schleife mit 1-Sample-Delay
f, g	Parallele Komposition	Zwei Signalprozessoren nebeneinander
$f : g$	Sequenzielle Komposition	Ausgang von f mit Eingang von g verbinden
$f < : g$	Split	Ein Ausgang auf mehrere Eingänge verteilen
$f > : g$	Merge	Mehrere Ausgänge auf einen Eingang zusammenführen

Abbildung 3.1 stellt dieselben Operatoren als Blockdiagramme dar.

Beispiel für sequenzielle Komposition: **process = + : abs;** verbindet die Ausgabe einer Addition mit einem Absolutwert. (Orlarey et al., 2009, S. 4)

Beispiel für rekursive Komposition: **process = + _;** erzeugt einen Integrator mit einer impliziten Verzögerung um ein Sample $Y(t) = X(t) + Y(t - 1)$. (Orlarey et al., 2009, S. 4)

Programm 3.2 zeigt einen Rauschgenerator mit rekursiver Pseudozufallszahlenerzeugung und einem Regler für die Signalamplitude (Orlarey et al., 2009, S. 5)

Die rekursive Definition erzeugt eine Pseudozufallsfolge. Die nachfolgenden Definitionen skalieren sie in den Bereich von minus eins bis eins und verknüpfen sie mit einem

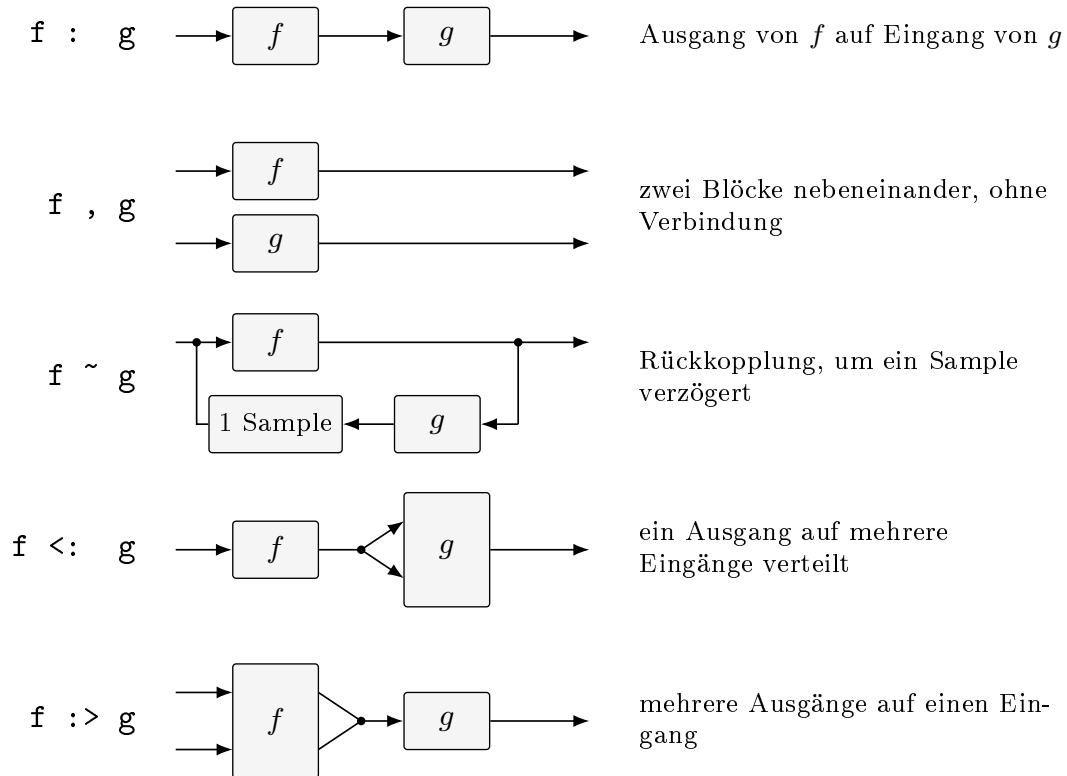


Abbildung 3.1: Die fünf Kompositionsoperatoren der Faust-Blockdiagramm

Programm 3.2: Rauschgenerator mit Lautstärkeregler in FAUST

```

1  random = +(12345) ~ *(1103515245);
2  noise = random / 2147483647.0;
3  process = noise * vslider("noise[style:knob]", 0, 0, 100, 0.1) / 100;
4

```

interaktiven Lautstärkeregler, um den resultierenden Signal zu steuern (Orlarey et al., 2009, S. 6).

3.4 Semantik

FAUST besitzt eine denotationale Semantik. Ein Programm beschreibt nicht eine Folge imperativer Einzelschritte, sondern die mathematische Bedeutung eines Signalprozessors. Digitale Signale werden dabei als diskrete Funktionen der Zeit aufgefasst. Ein Signalprozessor ist entsprechend eine Funktion, die Eingangs- in Ausgangssignale überführt (Orlarey et al., 2009, S. 2–4).

Diese Sichtweise trennt die Spezifikation eines DSP-Algorithmus von dessen konkreter Implementierung. Die FAUST-Quellbeschreibung legt fest, welche Signaltransforma-

Programm 3.3: Semantisch äquivalente FAUST-Beschreibungen

```

1    process = /(2) : @(10);
2    process = *(2) : @(7) : /(4) : @(3);
3

```

tion berechnet werden soll. Der Compiler entscheidet anschließend, wie sie effizient in C++ umgesetzt wird. Semantisch äquivalente Blockdiagramme können nach der Normalisierung dieselbe Implementierung erzeugen (Orlarey et al., 2009, S. 3, S. 12).

Für die vorliegende Arbeit ist diese Trennung wesentlich. Die FAUST-Variante und die manuelle C++-Referenz können unterschiedlich strukturiert sein. Beide müssen jedoch denselben Algorithmus und dieselbe Ein-/Ausgabesemantik realisieren. Der Vergleich bewertet daher die Qualität der abgeleiteten Implementierung und nicht lediglich Unterschiede in der Oberflächensyntax.

3.5 Compiler-Architektur

Der FAUST-Compiler verarbeitet eine Quellbeschreibung in mehreren aufeinander aufbauenden Analysephasen. Diese Phasen trennen die sprachliche Auswertung, die Ermittlung der mathematischen Bedeutung und die Optimierungsvorbereitung von der anschließenden C++-Codegenerierung (Orlarey et al., 2009, S. 10–13).

1. **Einlesen und Parsen der Quelldateien.** Der Compiler liest die Hauptdatei sowie über `import`-Direktiven eingebundene Dateien rekursiv ein. Das Ergebnis ist eine Menge benannter Definitionen. (Orlarey et al., 2009, S. 10–11).
2. **Auswertung der Blockdiagrammdefinitionen.** Algorithmische Definitionen werden ausgewertet und in eine explizite Blockdiagrammbeschreibung in Normalform überführt. Für die weitere Analyse liegen damit die primitiven Signaloperationen und ihre Verschaltung unmittelbar vor (Orlarey et al., 2009, S. 11–12).
3. **Symbolische Semantikanalyse und Normalisierung.** Der Compiler propagiert symbolische Eingangssignale durch das Blockdiagramm. Daraus leitet er für jedes Ausgangssignal eine mathematische Gleichung ab. Anschließend normalisiert er diese Gleichungen, sodass semantisch äquivalente Blockdiagramme dieselbe Repräsentation erhalten. Aufeinanderfolgende Verzögerungen können zusammengefasst werden. Konstante Faktoren können so angeordnet werden, dass Verzögerungsleitungen wiederverwendbar sind (Orlarey et al., 2009, S. 12).

Programm 3.3 zeigt zwei syntaktisch unterschiedliche Programme mit derselben Signalgleichung.

Beide Beschreibungen ergeben nach der Normalisierung dieselbe Ausgangsgleichung. Sie können deshalb auf dieselbe optimierte Implementierung abgebildet werden (Orlarey et al., 2009, S. 12).

4. **Typ- und Bereichsanalyse.** Für jede resultierende Signalgleichung ermittelt der Compiler den numerischen Datentyp, den möglichen Wertebereich, den Berechnungszeitpunkt, die Ausführungsgeschwindigkeit und die Parallelisierbarkeit. Konstante Signale können einmalig berechnet werden. Bedienelemente können block-

Programm 3.4: Zwischenspeicherung eines gemeinsam verwendeten Teilausdrucks

```

1  // Stereo zu Mono: kein Caching nötig
2  process = +;
3  → output0[i] = input0[i] + input1[i];
4
5  // Split: Ergebnis wird zwischengespeichert
6  process = + <: ,;
7  → float fTemp0 = input0[i] + input1[i];
8  output0[i] = fTemp0;
9  output1[i] = fTemp0;
10

```

weise und Audiosignale sampleweise verarbeitet werden. Diese Unterscheidungen sind Implementierungsdetails; aus Sicht der FAUST-Programmierung bleiben alle Werte reguläre Audiosignale (Orlarey et al., 2009, S. 12–13).

5. **Vorkommenanalyse.** Der Compiler untersucht, in welchen Kontexten Teilausdrücke auftreten und ob sie gemeinsam verwendet werden. Teure gemeinsame Teilausdrücke können in temporären Variablen zwischengespeichert werden. Dies gilt auch für Ausdrücke, die zwischen unterschiedlichen Geschwindigkeitskontexten benötigt werden. Nicht jeder mehrfach auftretende Teilausdruck wird jedoch automatisch ausgelagert (Orlarey et al., 2009, S. 13).

Erst nach diesen Phasen beginnt die Codegenerierung. Der Compiler übersetzt somit nicht die ursprüngliche Schreibweise des Programms, sondern eine analysierte und normalisierte Repräsentation seiner Signalverarbeitung (Orlarey et al., 2009, S. 13).

Erst nach diesen Phasen beginnt die Codegenerierung. Der Compiler übersetzt somit nicht die ursprüngliche Schreibweise des Programms, sondern eine analysierte und normalisierte Repräsentation seiner Signalverarbeitung (Orlarey et al., 2009, S. 13).

3.6 Codegenerierung

Auf Basis der analysierten Signalgleichungen erzeugt der FAUST-Compiler C++-Code. Die skalare Variante bildet die Grundlage. Der Vektor-Modus strukturiert diesen Code für die automatische Vektorisierung um. Der Parallel-Modus ergänzt den Vektor-Code um Direktiven für die nebenläufige Ausführung (Orlarey et al., 2009, S. 13–21).

3.6.1 Skalarer Modus

Im skalaren Modus berechnet eine Schleife die Ausgänge sampleweise. Für jede Addition $E_1 + E_2$ wird der Code von E_1 und E_2 generiert und mit einem $+$ verbunden. Wird das Ergebnis mehrfach verwendet, wird eine Zwischenspeicher-Variable angelegt, um redundante Berechnungen zu vermeiden (Orlarey et al., 2009, S. 13–14).

Programm 3.4 veranschaulicht den Unterschied zwischen einer direkten Berechnung und einer zwischengespeicherten Berechnung.

3.6.2 Vektor-Modus

Der Vektor-Modus erzeugt mehrere einfachere Schleifen, die über Vektoren kommunizieren. Dadurch erhält der nachgelagerte C++-Compiler eine Codeform, die die automatische Vektorisierung erleichtert. Gemeinsam verwendete, rekursive oder für Verzögerungsleitungen benötigte Ausdrücke können in eigenen Schleifen berechnet werden. Das Ergebnis ist ein gerichteter azyklischer Graph aus Berechnungsschleifen (Orlarey et al., 2009, S. 15–17).

Die automatische Vektorisierung unterscheidet vektorisierbare von nicht vektorisierbaren Ausdrücken anhand von Typ- und Kontextinformationen. Die dadurch erzielbare Beschleunigung hängt von Algorithmus, Compiler und Zielarchitektur ab und darf daher nicht als allgemeiner Leistungsgewinn interpretiert werden (Scaringella et al., 2003).

3.6.3 Parallel-Modus

Der Parallel-Modus baut auf dem Vektor-Modus auf und ergänzt den erzeugten C++-Code um Open Multi-Processing (OpenMP)-Direktiven. Unabhängige Schleifen desselben Ausführungsniveaus können parallel ausgeführt werden. Rekursive Abhängigkeiten begrenzen diese Parallelisierung (Orlarey et al., 2009, S. 17–21).

Spätere Arbeiten untersuchen zusätzlich dynamische Scheduling-Verfahren für den Schleifengraphen. Diese Verfahren ändern jedoch nichts daran, dass nutzbarer Parallelismus und der Aufwand der Synchronisierung von der Struktur des Signalprozessors abhängen (Letz et al., 2010).

3.6.4 Generierte Code

Der erzeugte C++-Code wird als Klasse ausgegeben. Zu den zentralen Methoden zählen die Abfrage der Ein- und Ausgänge, die Initialisierung, die Beschreibung der Benutzeroberfläche und die eigentliche Signalverarbeitung (Orlarey et al., 2009, S. 6–7).

- `getNumInputs()` / `getNumOutputs()` - Anzahl Ein-/Ausgänge
- `init(int samplingFreq)` - Initialisierung
- `buildUserInterface(UI*)` - GUI-Beschreibung
- `compute(int count, float** input, float** output)` - Signalverarbeitungsmethode

3.7 Backends und Architekturdateien

Architekturdateien trennen die Signalverarbeitungslogik von der Einbettung in ein Audio- oder Benutzeroberflächensystem. Sie umschließen die generierte C++-Klasse. Dadurch kann dieselbe FAUST-Quellbeschreibung für verschiedene Anwendungen und Plug-in-Formate verwendet werden (Orlarey et al., 2009, S. 8).

Die in diesem Abschnitt dargestellte Architekturauswahl stammt aus einer älteren FAUST-Version. Für aktuelle Zielsprachen, Zielplattformen und Architekturdateien ist deshalb die offizielle FAUST-Dokumentation maßgeblich (GRAME - Centre National de Création Musicale, 2025).

3.8 Performance

Die in der Grundquelle dargestellten Benchmarks vergleichen mehrere Testprogramme auf bestimmten Hardware- und Compilerkonfigurationen. Sie zeigen, dass der Nutzen von Vektorisierung und Parallelisierung von der Struktur des Signalprozessors, dem verfügbaren Parallelismus und der Speicherbandbreite abhängt (Orlarey et al., 2009, S. 22–28).

Die Messungen wurden mit einer historischen FAUST-Version sowie den damals verfügbaren Compilern und Testsystemen durchgeführt. Sie dienen deshalb der Einordnung der Compilerstrategien, erlauben jedoch keine Vorhersage für die in dieser Arbeit verwendete Hardware, Compilerkonfiguration oder Benchmark-Infrastruktur (Orlarey et al., 2009, S. 22–23).

- **Speicherbandbreitenbegrenzte Programme.** Im Benchmark `copy1` waren die skalare und die vektorisierte Variante ähnlich schnell. Die parallele Variante schnitt wegen der geteilten Speicherbandbreite schlechter ab (Orlarey et al., 2009, S. 22–23).
- **Programme mit begrenztem Parallelismus.** Für die Freeverb-Implementierung berichten die Autoren bei Verwendung des Intel-C++-Compilers auf zwei der drei Testsysteme moderate Zugewinne durch Vektorisierung und Parallelisierung (Orlarey et al., 2009, S. 24).
- **Programme mit hohem Parallelismus.** Für `karplus32` und `rms8` wurden mit dem Intel-C++-Compiler deutliche Leistungssteigerungen durch Vektorisierung und Parallelisierung beobachtet. Die Höhe des Gewinns blieb jedoch von Hardware und Speicherbandbreite abhängig (Orlarey et al., 2009, S. 24, S. 27–28)

3.9 Zusammenfassung

FAUST ist eine kompilierte Sprache für Audio-Signalverarbeitung mit formaler Semantik. Die Sprache trennt die Beschreibung eines Signalprozessors von dessen Implementierung und ermöglicht dem Compiler, daraus eigenständigen C++-Code zu erzeugen (Orlarey et al., 2009). Für die vorliegende Arbeit ist insbesondere relevant, dass dieselben DSP-Algorithmen sowohl als FAUST-Programm als auch als manuelle C++-Referenz implementiert und unter einer gemeinsamen Benchmark-Infrastruktur verglichen werden könne

Kapitel 4

Versuchsdesign und Benchmark-Methodik

Dieses Kapitel beschreibt, wie der Vergleich zwischen Faust-generiertem und manuell implementiertem C++-Code durchgeführt wird. Es legt fest, welche DSP-Bausteine untersucht werden, mit welchen Werkzeugen gemessen wird, auf welcher Plattform die Messungen laufen, welche Compilerkonfigurationen verwendet werden und welche Testdaten zum Einsatz kommen. Der Versuchsaufbau soll wiederholbar und für beide Implementierungen dieselben Bedingungen herstellen. Die konkrete Umsetzung im Quelltext beschreibt Kapitel 5, die Messergebnisse folgen in Kapitel 6.

4.1 Definition der untersuchten DSP-Bausteine

Für die Auswahl der Bausteine werden drei Kriterien herangezogen.

1. **Unterschiedliche Datenabhängigkeiten.** Die Bausteine sollen sich darin unterscheiden, ob und über wie viele Samples ein Ergebnis von früheren Ergebnissen abhängt. Gerade diese Abhängigkeiten bestimmen die Optimierungsmöglichkeiten des Compilers (Scaringella et al., 2003).
2. **Ein Skalierungsparameter je Baustein.** Jeder Baustein soll einen Parameter besitzen, über den sich der Rechenaufwand gezielt verändern lässt. Etwa die Blockgröße, die Filterlänge oder die Transformationslänge. Erst dadurch lässt sich beobachten, wie sich der Unterschied zwischen beiden Ansätzen mit dem Aufwand verändert.
3. **Nachweisbare Gleichwertigkeit.** Jeder Baustein muss klein genug sein, dass sich zeigen lässt, dass beide Implementierungen dasselbe berechnen.

Tabelle 4.1: Untersuchte DSP-Bausteine und ihre Eigenschaften

Baustein	Aufwand	Datenabhängigkeit	Skalierung
Gain	1 Multiplikation	keine	Blockgröße 64–1024
Akkumulator	1 Addition	Rückkopplung (1 Sample)	Blockgröße 64–1024
Biquad-Filter	5 Mult., 4 Add.	Rückkopplung (2 Samples)	Blockgröße 64–1024
FIR-Filter	L Mult., L Add.	keine	Filterlänge $L \in \{32, 64, 256\}$
FFT	$O(N \log_2 N)$	Butterfly-Graph	FFT-Größe $N \in \{64, 128, 256\}$

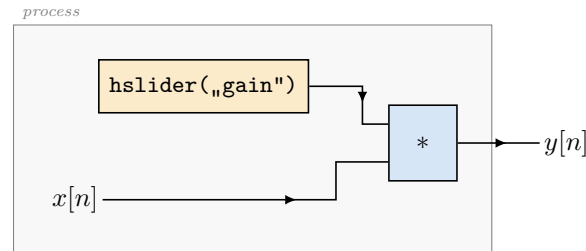


Abbildung 4.1: Verstärkungsstufe (`gain.dsp`). Beide Eingänge der Multiplikation liegen auf der linken Blockkante: das Audiosignal und das Bedienelement. Der Verstärkungsfaktor ist damit ein Signal und keine Übersetzungszeitkonstante; er wird je Aufruf von `compute()` aus dem Objekt gelesen. Zwischen aufeinanderfolgenden Samples besteht keine Abhängigkeit.

Tabelle 4.1 listet Baustein, Aufwand, Datenabhängigkeit, Skalierung gegenüber.

Die ersten vier Bausteine werden mit den Blockgrößen 64, 128, 256, 512 und 1024 Samples gemessen. Diese Größen entsprechen üblichen Puffergrößen in Audioanwendungen. Die FFT bildet eine Ausnahme. Bei ihr ist die Blockgröße an die Transformationslänge gebunden, weshalb nur die drei Längen 64, 128 und 256 gemessen werden. Der Grund für diese Begrenzung wird in Abschnitt 4.1.1 erläutert.

Der Akkumulator ist gegenüber der Aufzählung in Abschnitt 1.2 hinzugekommen. Er wird ergänzt, weil der Biquad-Filter Rückkopplung und Koeffizientenrechnung miteinander vermischt. Der Akkumulator trennt beides und misst die Rückkopplung allein.

Verstärkungsstufe (Gain)

Die Gain-Stufe skaliert jedes Sample mit einem konstanten Faktor $g = 0,5$ gemäß Gleichung 2.9. In Faust wird sie als Multiplikationen mit einem Bedienelement beschrieben. Der Vorgabewert beträgt 0,5. Es gibt keinen Zustand und keine Datenabhängigkeit zwischen den Samples. Jedes Ausgangssample kann unabhängig berechnet werden und damit sind die Daten vollständig parallel.

Der Baustein liefert zuerst den einfachsten Fall für den Laufzeitvergleich. Bei einer einzelnen Multiplikation pro Sample sollten beide Implementierungen praktisch denselben Maschinencode erzeugen. Zweitens dient er als Bezugspunkt für Auflösungsgrenze des Messaufbaus (Abschnitt 4.6). Zeigt sich hier ein Unterschied, so ist das zunächst ein Befund über die Messung und erst danach einer über Faust. Da die Operation zudem vollständig vektorisierbar ist, eignet sie sich als Überprüfung, ob die Codegenerierung die automatische Vektorisierung des C++-Compilers behindert.

Die Aussagekraft ist begrenzt. Ein Baustein mit einer Operation je Sample ist nicht rechenerisch, sondern Speicherzugriff begrenzt. Gemessen wird hier vor allem, wie schnell Werte geladen, geschrieben wird und wie gut gerechnet wird. Zur Beantwortung der Qualität der Codegenerierung ist die Verstärkungsstufe der schwächste Baustein.

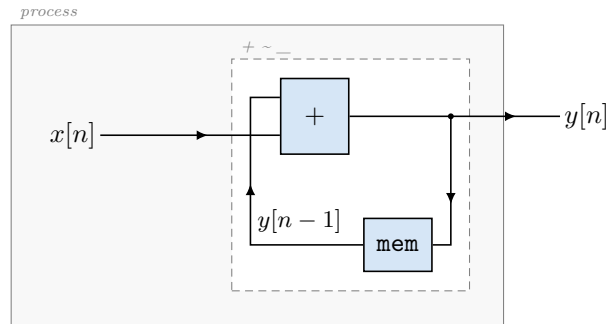


Abbildung 4.2: Akkumulator (`accum.dsp`, `process = + ~ _;`). Der innere Rahmen ist die rekursive Komposition. Wie in der Faust-Darstellung üblich ist der rückgekoppelte Teil gespiegelt gezeichnet: Sein Eingang liegt rechts, sein Ausgang links, sodass der Draht die Schleife schließt. Faust fügt die Verzögerung um ein Sample implizit ein; sie ist hier als `mem` sichtbar gemacht. Die Rückkopplung ist der einzige Inhalt des Bausteins, Koeffizienten kommen nicht vor.

Akkumulator (Accum)

Der Akkumulator berechnet

$$y[n] = x[n] + y[n - 1]. \quad (4.1)$$

Jedes Ausgangssample ist die Summe aller bisherigen Eingangssamples. In Faust entspricht das dem Rekursionsoperator aus Tabelle 3.1 in seiner kürzesten Form, also einer Addition mit Rückführung über ein Sample.

Abbildung 4.2 zeigt das zugehörige Faust-Blockdiagramm.

Der Baustein enthält weder Filterkoeffizienten noch weitere Logik und isoliert damit ausschließlich die Kosten der Rückkopplung. Er beantwortet keine eigenständige Frage, sondern trennt zwei mögliche Erklärungen. Zeigt sich beim Biquad-Filter ein Laufzeitunterschied zwischen Faust und C++, so lässt sich am Akkumulator prüfen, ob dieser aus der Rückkopplung selbst stammt oder aus der Art, wie die Koeffizienten verrechnet werden. Ohne diesen Vergleichsfall bliebe eine solche Beobachtung unentscheidbar.

Als Audio-Baustein ist der Akkumulator hingegen ohne praktischen Wert. Seine Ausgabe wächst unbegrenzt, und der aufsummierte Rundungsfehler nimmt mit der Blocklänge zu. Er dient rein als ein Diagnosefall und kein Anwendungsfall.

Biquad-Filter

Der Biquad-Filter ist ein Tiefpass zweiter Ordnung mit einer Grenzfrequenz von 800 Hz bei einer Abtastfrequenz von 48 kHz. Er verwendet die Koeffizienten $b_0 = b_2 = 0,00255054$, $b_1 = 0,00510107$, $a_1 = -1,85214649$ und $a_2 = 0,86234863$. Es wird darauf geachtet, dass die Gleichverstärkung 1 beträgt. Beide Implementierungen verwenden dieselben Koeffizienten und realisieren dieselbe Übertragungsfunktion nach Gleichung 2.11. Jedes Ausgangssample hängt von den beiden vorherigen Ein- und Ausgangswerten ab.

Abbildung 4.3 zeigt das zugehörige Faust-Blockdiagramm.

Der Baustein ist der aussagekräftigste der fünf, weil er zu drei Fragen zugleich beiträgt. Zur Laufzeit liefert er den Fall, in dem die Datenabhängigkeit die Optimierung

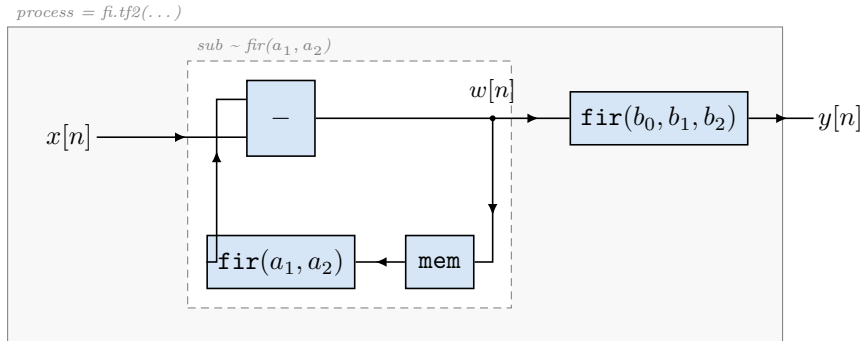


Abbildung 4.3: Biquad-Filter (`biquad.dsp`, `fi.tf2`). Die Bibliotheksfunktion ist eine sequenzielle Komposition aus einem Rekursionsblock und einem FIR-Block: `sub ~ fir(a1,a2) : fir(b0,b1,b2)`. Beide greifen auf dasselbe Zwischensignal w zu, weshalb nur eine Verzögerungskette entsteht – das ist die Direktform II. Die Direktform I der manuellen Implementierung (Abb. 4.4) ließe sich so nicht ausdrücken, weil ihr Vorwärtzweig über das Eingangs- und ihr Rückkopplungszweig über das Ausgangssignal läuft und sie deshalb zwei getrennte Ketten benötigt.

tatsächlich einschränkt, weil das Umordnen von Gleitkommaoperationen die Länge der Abhängigkeitskette verändern kann (Abschnitt 4.4). Zur numerischen Genauigkeit liefert er den einzigen Fall, in dem sich Rundungsfehler über die Rückkopplung fortpflanzen statt sich nur zu addieren.

Die manuelle Implementierung folgt der Direktform I. Sie ist die wörtliche Übertragung von Gleichung 2.11. Jedes Ausgangssample wird unmittelbar aus den beiden vorherigen Eingangs- und den beiden vorherigen Ausgangswerten berechnet. Der Filter speichert deshalb vier Werte.

Faust rechnet in zwei Schritten. Zuerst entsteht aus dem Eingang und zwei zurückgeführten Werten ein Zwischensignal w , aus dem anschließend das Ausgangssignal gebildet wird:

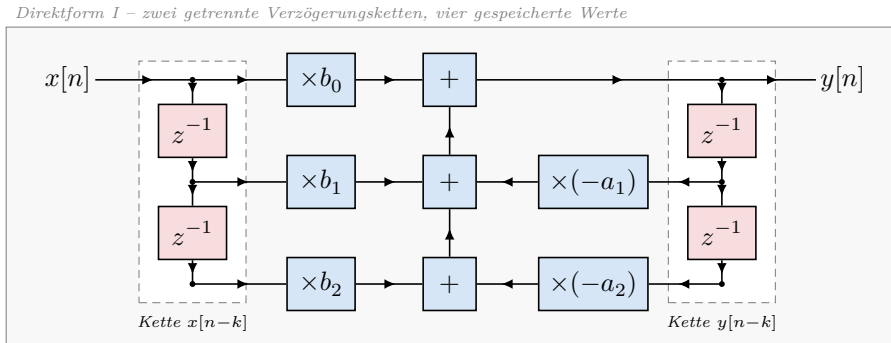
$$w[n] = x[n] - a_1 w[n-1] - a_2 w[n-2], \quad (4.2)$$

$$y[n] = b_0 w[n] + b_1 w[n-1] + b_2 w[n-2]. \quad (4.3)$$

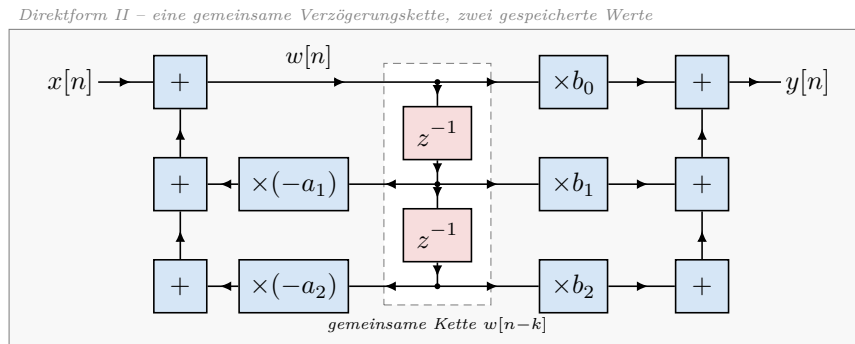
Das ist die Direktform II. Sie speichert zwei Werte weil beide Schritte dieselbe Verzögerungskette benutzen.

Für Faust ist diese Form die naheliegende. Der Rekursionsoperator aus Tabelle 3.1 beschreibt genau den ersten Schritt, und der zweite Schritt ist ein gewöhnlicher FIR-Filter. Die Bibliotheksfunktion `fi.tf2` ist deshalb aus diesen beiden Teilen zusammengesetzt. Die Direktform I ließe sich so nicht ausdrücken. Der Vorwärtzweig läuft über das Eingangs- und der Rückkopplungszweig über das Ausgangssignal. Es wären also zwei getrennte Verzögerungsketten nötig.

Der Unterschied ist bewusst. Jede Seite verwendet die Formulierung, die für sie die naheliegende ist, denn genau dieser Unterschied gehört zu dem, was der Vergleich sichtbar machen soll. Die Direktform I schiebt je Sample vier Werte um, die Direktform II nur zwei. Die unterschiedliche Formulierung kann einen gemessenen Laufzeitunterschied



(a) Direktform I: getrennte Ketten für Eingangs- und Ausgangswerte



(b) Direktform II: beide Zweige teilen sich dieselbe Kette

Abbildung 4.4: Die beiden Realisierungsformen des Biquad-Filters. Beide berechnen dieselbe Übertragungsfunktion und liefern – bis auf Rundung – dasselbe Ausgangssignal, unterscheiden sich aber im Zustand. Die Direktform I (a) ist die wörtliche Übertragung der Differenzengleichung (Gleichung 2.11): Sie führt eine eigene Kette für die vergangenen Eingangs- und eine zweite für die vergangenen Ausgangswerte und speichert deshalb vier Werte. Die Direktform II (b) zieht die Rekursion vor und lässt beide Zweige auf dasselbe Zwischensignal w zugreifen; sie kommt mit einer Kette und zwei gespeicherten Werten aus. Die manuelle Implementierung folgt der Form (a) (Abb. 4.4a), die Faust-Bibliotheksfunktion `fi.tf2` erzeugt die Form (b) (Abb. 4.4b). Die Verzögerungsglieder sind zur Hervorhebung abgesetzt gezeichnet.

verursachen und nicht nur von der Codegenerierung. Mit dem vorliegenden Aufbau lassen sich beide Ursachen nicht trennen. Dass beide Varianten dennoch dasselbe Ergebnis liefern, wird in Abschnitt 5.5 nachgewiesen.

Der Unterschied ist bewusst. Jede Seite verwendet die Formulierung, die für sie die naheliegende ist, denn genau dieser Unterschied gehört zu dem, was der Vergleich sichtbar machen soll. Abbildung 4.4 stellt beide Formen gegenüber. Die unterschiedliche Formulierung kann einen gemessenen Laufzeitunterschied verursachen, der dann nicht der Codegenerierung zuzuschreiben ist. Mit dem vorliegenden Aufbau lassen sich beide Ursachen nicht trennen. Dass beide Varianten dennoch dasselbe Ergebnis liefern, wird in Abschnitt 5.5 nachgewiesen.

FIR-Filter

Untersucht werden FIR-Filter mit $L \in \{32, 64, 256\}$ Koeffizienten. Die Koeffizienten bilden eine normierte Rampe

$$b_k = \frac{2(k+1)}{L(L+1)}, \quad k = 0, \dots, L-1, \quad (4.4)$$

deren Summe eins ergibt.

In Faust wird der Filter über die Bibliotheksfunktion `fi.fir` mit einer erzeugten Koeffizientenliste beschrieben. Jedes Ausgangssample ist die gewichtete Summe der letzten L Eingangssamples. Eine Rückkopplung gibt es nicht. Die manuelle Implementierung hält die vergangenen Eingangswerte in einem Puffer. Ihre Umsetzung zeigt Abschnitt 5.1.

Abbildung 4.5 zeigt das zugehörige Faust-Blockdiagramm.

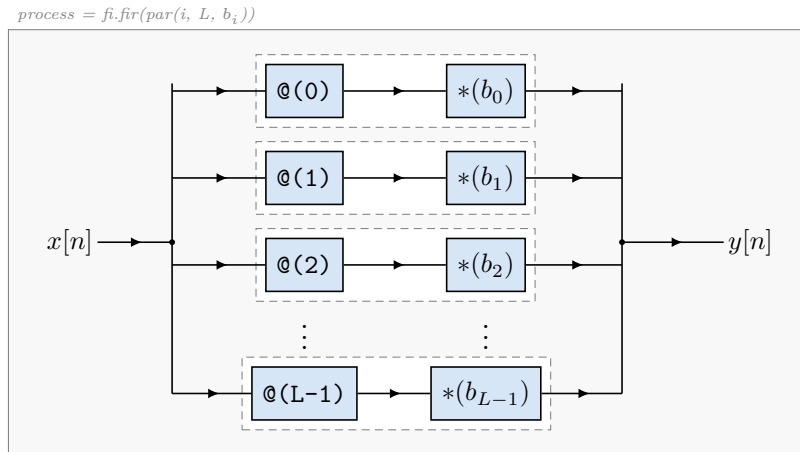


Abbildung 4.5: FIR-Filter (`fir32.dsp` u.a., `fi.fir`). Das Eingangssignal wird auf L Zweige verteilt ($<:$); jeder Zweig ist eine sequenzielle Komposition aus Verzögerung und Gewichtung, danach werden alle Zweige zusammengeführt ($:>$). Gezeichnet sind die ersten drei und der letzte Zweig; gemessen wird mit $L \in \{32, 64, 256\}$. Die Struktur ist vollständig parallel – es gibt keine Rückkopplung, und der Faust-Compiler schreibt die Summe im erzeugten C++-Code als eine einzige ausgeschriebene Zeile aus (Abb. 5.2).

Der Baustein trägt zur Beantwortung der Frage nach dem Skalierungsverhalten bei. Er ist der einzige, bei dem sich der Rechenaufwand unabhängig von der Blockgröße verändern lässt, und drei Filterlängen zeigen, ob ein Unterschied zwischen beiden Ansätzen mit dem Aufwand wächst, schrumpft oder gleich bleibt. Da die Faltung ohne Rückkopplung auskommt, ist sie zugleich der Fall, an dem sich der Nutzen der automatischen Vektorisierung am deutlichsten zeigen sollte. Schließlich ist er für die Binärgröße aufschlussreich, weil der Faust-Compiler die Faltung vollständig ausrollt und die Codegröße daher mit L wachsen dürfte.

4.1.1 Fast-Fourier-Transformation

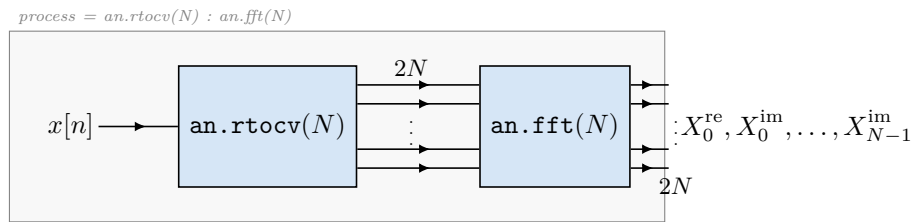
Untersucht werden FFTs mit $N \in \{64, 128, 256\}$. Beide Implementierungen berechnen eine gleitende Fenster-FFT: Für jedes Eingangssample wird die N -Punkt-FFT über die letzten N Samples berechnet. Der Baustein besitzt einen Eingang und $2N$ Ausgänge, die Real- und Imaginärteil abwechselnd ausgeben.

In Faust wird er über die Bibliotheksfunktionen `an.rtcv` und `an.fft` beschrieben. Die manuelle Implementierung ist eine iterative Radix-2-FFT nach Cooley-Tukey mit Dezimierung im Zeitbereich (Cooley & Tukey, 1965). Ihren Aufbau zeigt Abschnitt 5.1.

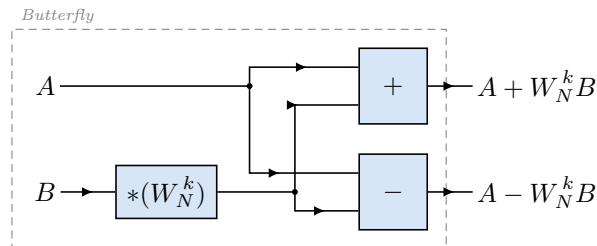
Abbildung 4.6 zeigt das zugehörige Faust-Blockdiagramm.

Die FFT ist der einzige Baustein, bei dem der Aufwand nicht linear wächst, und ergänzt die Skalierungsfrage. Vor allem aber ist sie der Fall, in dem sich die Codegenerierung am stärksten von gewöhnlichem C++ unterscheidet. Der Faust-Compiler rollt die gesamte Butterfly-Struktur aus, während die manuelle Implementierung sie in verschachtelten Schleifen durchläuft. Damit ist die FFT der aussagekräftigste Baustein für die Fragen nach Binärgröße und Speicherbedarf und zugleich der einzige, an dem die Übersetzungsdauer des Faust-Compilers spürbar wird.

Der Vergleich hat hier zwei Schwächen. Die gleitende Fenster-FFT berechnet für



(a) Gesamtstruktur: ein Eingang, $2N$ Ausgänge



(b) Elementare Butterfly-Operation; $\log_2 N$ Stufen zu je $N/2$ Stück

Abbildung 4.6: FFT (`fft.dsp`). `an.rtcv` bildet aus dem Eingangssignal ein gleitendes Fenster der Länge N und serialisiert es – mit dem neuesten Sample zuerst – als $2N$ Signale, in denen Real- und Imaginärteil abwechseln. `an.fft` transformiert dieses Fenster. Anders als beim FIR-Filter ist die innere Struktur in Faust nicht als Schleife formuliert, sondern als Blockdiagramm aus $\log_2 N$ Stufen zu je $N/2$ Butterflies, das der Compiler vollständig ausrollt. Für $N = 256$ sind das 1024 Butterflies je Ausgangssample. Darin liegt der wesentliche Unterschied zur manuellen Implementierung, die dieselbe Struktur in verschachtelten Schleifen durchläuft.

jedes einzelne Sample eine vollständige Transformation und ist damit weit aufwendiger als die in der Praxis übliche Blockverarbeitung mit Fenstervorschub. Diese Form ergibt sich aus der Semantik der verwendeten Faust-Funktion und wurde nicht gewählt, weil sie praxisnah wäre. Zudem sind die untersuchten Transformationslängen klein. Die Zwischenspeicher der Testplattform fassen alle drei Größen mühelos, sodass sich Effekte der Speicherhierarchie nicht zeigen.

Zwei weitere Festlegungen sind wesentlich. Erstens wird bewusst keine Fremdbibliothek verwendet. Eine frühere Fassung des Versuchsaufbaus nutzte FFTW. Dieser Vergleich hätte Faust gegen eine handoptimierte Bibliothek mit Assembler-Kernen gestellt und nicht gegen manuell geschriebenen C++-Code. Zweitens serialisiert Faust das Analysefenster mit dem neuesten Sample zuerst. Die manuelle Implementierung und die Referenzimplementierung folgen dieser Reihenfolge, weil andernfalls beide Varianten unterschiedliche Spektren berechnen würden.

Grenzen der Auswahl

Die fünf Bausteine decken nur einen schmalen Ausschnitt dessen ab, was in der Audio-Signalverarbeitung vorkommt. Alle sind linear, zeitinvariant und arbeiten mit konstanten Koeffizienten. Nicht untersucht werden nichtlineare Verarbeitung, zeitvariable Parameter, Mehrkanalverarbeitung sowie Bausteine mit großen Verzögerungsspeichern, wie sie etwa in Nachhallalgorithmen auftreten. Ebenso wenig untersucht wird eine vollständige Signalkette, in der mehrere Bausteine hintereinander laufen. Die Ergebnisse gelten deshalb für die beschriebenen Bausteine auf der beschriebenen Plattform.

Eine zweite Einschränkung betrifft die Vergleichsbasis selbst. Die manuellen Implementierungen sind im Rahmen dieser Arbeit entstanden und nicht von Personen geschrieben, die auf handoptimierten DSP-Code spezialisiert sind. Ein erfahrener Entwickler könnte die C++-Seite an mehreren Stellen weiter optimieren. Umgekehrt bleibt auf der Faust-Seite der Vektor-Modus ungenutzt. Der Vergleich betrifft daher jeweils die naheliegende Umsetzung beider Wege und nicht deren jeweiliges Optimum. Abschnitt 7.3 geht darauf erneut ein.

4.2 Benchmark-Tools und Messgrößen

4.2.1 Messwerkzeuge

Die Laufzeitmessung erfolgt ausschließlich mit Google Benchmark. Die Bibliothek ist als Submodul eingebunden und wird gemeinsam mit dem Projekt übersetzt. Aus einem einzigen Messlauf werden sowohl die Mittelwerte mit Fehlerbalken als auch Median, 95. Perzentil und Interquartilsabstand abgeleitet.

Jeder Baustein wird mit 100 Wiederholungen gemessen. Die Option `report_aggregates_only` wird dabei auf `false` gesetzt. Sie sorgt dafür, dass die einzelnen Wiederholungen und nicht nur deren Aggregate in die JSON-Ausgabe gelangen. Erst dadurch lassen sich Median, Perzentile und die statistischen Tests aus Abschnitt 4.6 berechnen.

Innerhalb der Messschleife verhindern die Hilfsfunktionen `DoNotOptimize` und `ClobberMemory`, dass der Compiler den Aufruf als toten Code entfernt (Google LLC, 2023). Die Messprozesse werden mit `taskset` an einen festen CPU-Kern gebunden. Das verhindert

Kernwechsel während der Messung und sorgt dafür, dass die schnellen Caches erhalten bleiben (Mytkowicz et al., 2009).

Gemessen wird über die Blockgrößen $N \in \{64, 128, 256, 512, 1024\}$. Für die FFT wird je Transformationslänge ein eigenes ausführbares Programm erzeugt, weil der Faust-Compiler die FFT je Größe vollständig ausrollt und die generierte Klasse damit größen-spezifisch ist.

Es existiert ein zweiter, unabhängiger Messstand auf Basis von `std::chrono::steady_clock`. Er misst jeden einzelnen `compute()`-Aufruf getrennt, wärmt vor jeder Runde beide Co-depfade vollständig auf und wechselt die Reihenfolge von Faust und C++ rundenweise. Dies gilt nur als Gegenprobe.

4.2.2 Messgrößen

Tabelle 4.2 fasst die erhobenen Messgrößen zusammen.

Tabelle 4.2: Erhobene Messgrößen und ihre Ermittlung

Messgröße	Ermittlung	Werkzeug
CPU-Zeit je Block	Median über 100 Wiederholungen	Google Benchmark
Streuung	95. Perzentil, Interquartilsabstand	Auswertungsskript
Durchsatz	verarbeitete Samples je Sekunde	Google Benchmark
Binärgröße	gestrippte Größe minus Basislinie	<code>strip, stat</code>
Zustandsgröße	<code>sizeof</code> der DSP-Klasse	Probe-Programm
Speicherbedarf	maximaler RSS, Heap-Spitze	GNU <code>time</code> , Valgrind
Genauigkeit	Fehler gegenüber float64-Referenz	Vergleichsprogramm
Codelänge	Codezeilen je Implementierung	Auswertungsskript
Compile-Zeit	Laufzeit des Faust-Compilers	Messskript

Die wichtigste Messgröße ist die Prozessorzeit pro Audioblock. Zusätzlich werden Speicherverbrauch und Binärgröße erfasst. Diese Werte beeinflussen die praktische Nutzbarkeit einer Implementierung neben der reinen Laufzeit. Codelänge und Implementierungskomplexität ergänzen den quantitativen Vergleich um Aspekte der Entwicklererfahrung. Die genaue Beschreibung dieser Größen, die Zahl der Wiederholungen und die statistische Auswertung stehen in Kapitel 4.

Die *Binärgröße* wird nicht als absolute Dateigröße berichtet. Ein übersetztes Programm besteht zu hauptsächlich aus Startup-Code und Laufzeit-Code der Standardbibliothek. Gemessen wird deshalb die Differenz zwischen einem Probe-Programm, das genau einen Baustein enthält, und einer Basislinie mit leerer `main`-Funktion. Beide werden vorher gestrippt. Für die FIR-Filter existiert je Filterlänge ein eigenes Probe-Programm, damit nicht alle drei generierten Klassen in dasselbe Binary gelangen.

Der *Speicherbedarf* wird auf drei Ebenen erfasst. Die Zustandsgröße `sizeof` beschreibt den Speicher, den die DSP-Klasse selbst belegt. Der maximale Resident-Set-Size-Wert und die Heap-Spitze beschreiben den Speicherbedarf des laufenden Prozesses. Von beiden wird die Basislinie abgezogen. Zusätzlich zählt das Probe-Programm über einen überladenen `new`-Operator die Speicheranforderungen innerhalb von `compute()`. Diese Zahl muss null sein. Eine Speicheranforderung im Audio-Pfad ist mit Echtzeitbetrieb wirkt sich sehr negativ aus, weil der Allocator Sperren halten, Speicher beim

Betriebssystem nachfordern und Verfahren mit unbeschränkter Laufzeit verwenden kann (Buttazzo, 1997).

Die *numerische Genauigkeit* wird als maximaler und mittlerer Absolutfehler sowie als quadratisches Mittel des Fehlers gegenüber einer Referenz in doppelter Genauigkeit angegeben. Der relative Fehler wird auf den größten Betrag der Referenz bezogen und als Vielfaches der Maschinengenauigkeit von `float` ausgedrückt, also von $2^{-23} \approx 1,19 \cdot 10^{-7}$. Der Effektivwert wäre hier ein ungeeigneter Bezugswert: Bei abklingenden Signalen wie einer Impulsantwort sinkt er mit wachsender Signallänge, während der absolute Fehler konstant bleibt. Die Fehlerquote stiege dann scheinbar an, obwohl sich nichts verschlechtert hat.

Die *Codelänge* wird als Zahl der Codezeilen gezählt. Als Codezeile gilt eine Zeile, die weder leer ist noch ausschließlich aus einem einzeiligen Kommentar besteht. Gezählt wird getrennt nach der Faust-Quelldatei, dem zugehörigen C++-Wrapper und der manuellen Implementierung. Die Trennung ist nötig, weil der Wrapper für alle Bausteine nahezu identisch ist und damit nicht zur eigentlichen Beschreibungsleistung gehört.

Die *Compile-Zeit* des Faust-Compilers wird ergänzend erhoben. Dazu werden aus Vorlagen Programme mit wachsender Rekursionstiefe, wachsender Baumtiefe und wachsender FFT-Größe erzeugt und übersetzt. Läufe mit einer Übersetzungsdauer über 30 Sekunden werden abgebrochen.

4.3 Hardwareumgebung

Alle Messungen laufen auf der in Tabelle 4.3 beschriebenen Plattform. Die Umgebung wird bei jedem Messlauf automatisch protokolliert. Das Protokoll enthält neben Hardware und Werkzeugversionen. Zusätzlich wird die Systemlast zum Zeitpunkt der Messung dokumentiert. Als Richtwert für belastbare Messungen gilt eine Ein-Minuten-Last unter 0,3.

Tabelle 4.3: Messumgebung

Komponente	Ausprägung
Prozessor	AMD Ryzen 7 4700U, 8 Kerne, 1 Thread je Kern
Cache	L1d 256 KiB, L2 4 MiB, L3 4 MiB
Betriebssystem	Ubuntu 24.04 unter WSL2, Kernel 6.18
C++-Compiler	g++ 13.3.0
Build-System	CMake 3.29.3, Ninja 1.11.1
Faust-Compiler	Faust 2.83.1
Auswertung	Python 3.12.3, NumPy 2.5.1

Die Wahl der Plattform bringt zwei Einschränkungen mit sich, die bei der Interpretation der Ergebnisse zu berücksichtigen sind. Erstens laufen die Messungen in einer virtuellen Maschine unter WSL2. Zweitens lässt sich der Frequenzregler des Prozessors in dieser Umgebung nicht auslesen und damit auch nicht festsetzen. Der verwendete Prozessor ist ein Mobilprozessor, dessen Taktfrequenz von Temperatur und Energiebudget abhängt.

Für den Versuchsaufbau wurden drei Vorkehrungen vorgenommen. Beide Implementierungen werden innerhalb desselben Prozesses und desselben Zeitfensters gemessen,

die Messungen werden an einen festen Kern gebunden, und es wird eine hohe Zahl von Wiederholungen ausgewertet. Berichtet werden deshalb Verhältnisse zwischen beiden Implementierungen und keine absoluten Zeiten als Plattformkennwerte. Abschnitt 7.3 greift diese Einschränkung auf.

4.4 Compilerkonfigurationen

4.4.1 Aufruf des Faust-Compilers

Der Faust-Compiler wird für alle Bausteine gleich aufgerufen, nämlich mit der Option `-cn` für den Namen der erzeugten Klasse und ohne weitere Schalter.

Verwendet wird damit das voreingestellte C++-Backend im skalaren Modus. Die Optionen für den Vektor-Modus und den Parallel-Modus werden bewusst nicht gesetzt. Die Forschungsfrage vergleicht den generierten C++-Code mit manuell geschriebenem C++-Code. Würde auf der Faust-Seite zusätzlich der Vektor-Modus oder OpenMP aktiviert, vergliche man zwei unterschiedliche Parallelisierungsstrategien statt zweier Beschreibungswege. Die Wirkung dieser Modi ist zudem bereits Gegenstand eigener Untersuchungen (Letz et al., 2010; Scaringella et al., 2003). Die automatische Vektorisierung durch `g++` bleibt für beide Seiten aktiv.

4.4.2 Flag-Matrix

Beide Implementierungen werden immer mit identischen Compilerflags übersetzt. Für die Untersuchung des Einflusses der Compileroptimierung wird die in Tabelle 4.4 gezeigte Matrix durchlaufen. Jede Konfiguration erhält ein eigenes Build-Verzeichnis, sodass keine Zwischenergebnisse aus einer anderen Konfiguration wiederverwendet werden.

Tabelle 4.4: Untersuchte Compilerkonfigurationen

Bezeichnung	Flags	Zweck
O0	-O0	unoptimierter Bezugspunkt
O1	-O1	Grundoptimierungen
O2	-O2	üblicher Standard
Os	-Os	Optimierung auf Codegröße
O3	-O3	Vektorisierung aktiv
O3_native	-O3 -march=native	Befehlssatz der Hardware
O3_native_fast	-O3 -march=native -ffast-math	gelockerte FP-Regeln

Alle Konfigurationen enthalten zusätzlich `-DNDEBUG`. Als Sprachstandard wird C++17 verwendet. Die Optimierungsflags werden im Build-System als erzwungener Cache-Eintrag gesetzt, damit der zwischengespeicherte Wert nicht von den tatsächlich verwendeten Flags abweichen kann.

Die Aufnahme von `-ffast-math` aktiviert unter anderem `-fassociative-math` und erlaubt dem Compiler damit, Gleitkommaadditionen unter Missachtung der IEEE-754-Semantik frei umzuordnen (*Using the GNU Compiler Collection (GCC), Version 13.3.0*, 2024). Bei rückgekoppelten Filtern kann das die Abhängigkeitskette verlängern und die Laufzeit erhöhen statt sie zu senken (Parhi, 1999).

4.5 Testdatengenerierung

Alle Testsignale werden durch ein Python-Skript erzeugt und als binäre Rohdateien im Format `float32` abgelegt. Als Zufallsgenerator dient der Standardgenerator von NumPy mit dem festen Startwert 42. Ziel ist es über beliebig viele Läufe und über beide Implementierungen hinweg wertgleiche Signale zu verwenden. Faust-Variante und C++-Referenz lesen dieselbe Datei.

Es werden zwei Kategorien von Testdaten erzeugt.

Signale für die Laufzeitmessung

Für die Blockgrößen 64, 128, 256, 512, 1024 und 2048 Samples wird jeweils ein Signal aus gleichverteiltem Rauschen im Wertebereich $[-1; 1]$ erzeugt.

Signale für den Genauigkeitsvergleich

Für den Genauigkeitsvergleich werden die in Tabelle 4.5 aufgeführten Signaltypen in den Längen 64, 256, 1024 und 4096 Samples erzeugt. Sie decken typische Audiosignale ebenso ab wie Randfälle.

Tabelle 4.5: Signaltypen für den Genauigkeitsvergleich

Typ	Definition	Zweck
impulse	Einheitsimpuls	Impulsantwort der Filter
step	Sprungfunktion	Sprungantwort, Einschwingverhalten
silence	Nullsignal	trivialer Randfall
dc	Konstantwert 0,5	einfachste Linearitätsprüfung
sine	1-kHz-Sinus bei 48 kHz	typisches Audiosignal
extreme	alternierend $\pm 1,0$	ungünstigster Fall für Rundung
noise	Rauschen, eigener Startwert	unabhängiger Zufallsfall

Referenz in doppelter Genauigkeit

Ein Vergleich, der ausschließlich Faust gegen die C++-Referenz stellt, kann einen systematischen Fehler nicht aufdecken, den beide Implementierungen gemeinsam machen. Deshalb wird zusätzlich für jeden Baustein und jedes Testsignal eine Referenzausgabe in doppelter Genauigkeit mit NumPy berechnet und als Rohdatei abgelegt. Die Referenzalgorithmen bilden die Struktur der C++-Bausteine nach und unterscheiden sich von diesen nur im Datentyp. Beide Implementierungen werden anschließend gegen diese Referenz gemessen.

4.6 Auswertungsverfahren

4.6.1 Kennzahlen und statistische Prüfung

Aus den Einzelwiederholungen jedes Bausteins wird der Median der CPU-Zeit gebildet. Der Vergleich beider Implementierungen erfolgt über das Verhältnis

$$r = \frac{\tilde{t}_{\text{Faust}}}{\tilde{t}_{\text{C++}}}, \quad (4.5)$$

wobei $\tilde{t}$ jeweils den Median bezeichnet. Ein Wert $r < 1$ bedeutet, dass die Faust-Variante schneller ist.

Hierzu wird mit dem Bootstrap-Verfahren gearbeitet. (Efron & Tibshirani, 1993). Aus den 100 gemessenen Werten werden zufällig wieder 100 gezogen, wobei derselbe Wert mehrfach gezogen werden darf. Für diese neue Stichprobe wird das Verhältnis erneut berechnet. Nach 10000 Wiederholungen liegen 10000 Verhältnisse vor. Die mittleren 95 Prozent davon bilden das Konfidenzintervall. Der Startwert des Zufallsgenerators ist fest, damit das Intervall reproduzierbar bleibt.

Ergänzend wird der Mann-Whitney-U-Test gerechnet (Mann & Whitney, 1947). Er vergleicht nicht die Messwerte selbst, sondern ihre Reihenfolge: Alle 100 Werte werden der Größe nach sortiert, und der Test prüft, ob die Werte der einen Seite systematisch weiter vorne liegen. Weil nur die Reihenfolge zählt, verschiebt ein einzelner Ausreißer das Ergebnis kaum. Das passt zu Laufzeitmessungen, die nach unten durch die Hardware begrenzt sind, nach oben aber durch das Betriebssystem beliebig weit gestreckt werden können.

Das Intervall beschreibt allerdings nur, wie stark das Verhältnis schwanken würde, wenn die Messung unter identischen Bedingungen wiederholt würde. Da jede der 100 Wiederholungen bereits ein Mittelwert über viele Aufrufe ist, liegen diese Werte eng beieinander, und die Intervalle fallen entsprechend schmal aus. Ein schmales Intervall belegt daher eine gut reproduzierbare Messung und nicht, dass das Verhältnis den wahren Leistungsunterschied genau trifft. Die Wahl der Compilerflags, die unterschiedliche Filterstruktur oder die Virtualisierung der Testplattform gehen in das Intervall nicht ein.

4.6.2 Auflösungsgrenze des Messaufbaus

Ein signifikanter Unterschied ist nicht automatisch ein inhaltlich bedeutsamer Unterschied. Bei einer hohen Zahl von Wiederholungen wird auch reines Messrauschen signifikant. Der Versuchsaufbau bestimmt deshalb eine Auflösungsgrenze und kennzeichnet alle Effekte unterhalb dieser Grenze als nicht auflösbar.

Als Bezugspunkt dient die Verstärkungsstufe. Die Annahme lautet, dass beide Implementierungen für diesen Baustein praktisch identischen Code erzeugen, die dort gemessene Abweichung also Messrauschen darstellt. Der größte bei diesem Baustein gemessene Abstand $|r - 1|$ über alle Blockgrößen wird als Auflösungsgrenze verwendet. Die Annahme ist Teil des Verfahrens und kein Ergebnis. Sie ist in Kapitel 6 anhand des generierten Codes und der gemessenen Werte zu prüfen.

Aus Verhältnis, Konfidenzintervall und Auflösungsgrenze ergibt sich für jeden Messpunkt eine von drei Aussagen:

- Enthält das Konfidenzintervall den Wert eins, ist kein Unterschied nachweisbar.
- Liegt $|r-1|$ unterhalb der Auflösungsgrenze, ist der Unterschied mit diesem Aufbau nicht auflösbar.
- Andernfalls wird der Unterschied als Prozentwert berichtet.

4.7 Übersicht der Messreihen

Aus den bisherigen Festlegungen ergeben sich neun Messreihen. Sie laufen als zusammenhängende Kette ab und werden nicht einzeln von Hand angestoßen. Tabelle 4.6 ordnet jeder Reihe zu, was sie erhebt und zu welcher Teilfrage aus Abschnitt 1.1 sie beiträgt.

Tabelle 4.6: Messreihen und ihr Beitrag zur Beantwortung der Teilfragen

Messreihe	Erhebt	Beitrag
Umgebungsprotokoll	Hardware, Werkzeuge, Git-Stand	Voraussetzung aller Reihen
Referenzausgaben	Sollwerte in doppelter Genauigkeit	Grundlage der Genauigkeitsprüfung
Genauigkeitsprüfung	Fehler beider Varianten	Genauigkeit; Gültigkeitsnachweis
Laufzeitmessung	CPU-Zeit je Block, fünf Blockgrößen	Laufzeit und Skalierung
Optimierungssweep	Dieselbe Messung, sieben Flagsätze	Einfluss der Compileroptimierung
Binärgrößenmessung	Gestrippte Größe je Baustein	Binärgröße
Speichermessung	Zustand, RSS, Heap, Allokationen	Speicherverbrauch
Quelltextumfang	Codezeilen je Variante	Codelänge
Übersetzungsdauer	Laufzeit des Faust-Compilers	Ergänzend, keine Teilfrage

Die Reihenfolge ist nicht beliebig. Zuerst entstehen die Referenzausgaben, danach wird die Genauigkeit geprüft, und erst zuletzt wird die Laufzeit gemessen. Die folgenden Abschnitte beschreiben für jede Reihe, welche Behauptung sie prüft, welches Ergebnis sie liefern soll und welche Frage damit beantwortet wird.

4.7.1 Voraussetzungen

Die ersten drei Reihen liefern keine Ergebnisse zur Forschungsfrage. Sie stellen sicher, dass die übrigen Zahlen überhaupt eine Bedeutung haben.

Das *Umgebungsprotokoll* prüft die Behauptung, dass die Messungen wiederholbar sind. Es hält Hardware, Compiler-, Faust- und NumPy-Version, den Git-Stand des Repositories und die Systemlast fest und warnt, wenn zum Zeitpunkt der Messung nicht

eingescheckte Änderungen vorliegen.

Die *Referenzausgaben* in doppelter Genauigkeit liefern die Sollwerte für jeden Baustein und jedes Testsignal. Sie sind nötig, weil ein Vergleich, der nur Faust gegen die C++-Referenz stellt, einen Fehler nicht aufdecken kann, den beide Seiten gemeinsam machen. Erst eine unabhängig gerechnete Referenz macht solche Fälle sichtbar.

Die *Genauigkeitsprüfung* prüft die zentrale Voraussetzung des gesamten Versuchs, nämlich dass beide Implementierungen dieselbe Funktion berechnen. Als Ergebnis liefert sie den maximalen und den mittleren Absolutfehler sowie das quadratische Mittel des Fehlers beider Varianten gegenüber der Referenz, bezogen auf die Auflösung von `float`. Widerlegt wäre die Voraussetzung, wenn ein Fehler deutlich oberhalb dieser Auflösung läge. Dann rechnen beide Seiten nicht dasselbe, und jeder Laufzeitvergleich wäre gegenstandslos. Aus diesem Grund bricht die Messkette an dieser Stelle ab, statt mit der Laufzeitmessung fortzufahren. Zugleich beantwortet die Reihe die Teilfrage nach der numerischen Genauigkeit.

4.7.2 Laufzeit und Compilerverhalten

Die beiden folgenden Reihen beantworten den Kern der Forschungsfrage.

Die *Laufzeitmessung* prüft die Behauptung, dass Faust-generierter Code einer manuell implementierten C++-Referenz mindestens ebenbürtig ist. Als Ergebnis liefert sie die Median-CPU-Zeit je Block für fünf Blockgrößen und alle Bausteine, dazu das Verhältnis beider Implementierungen mit Konfidenzintervall und Signifikanztest. Über die drei Filterlängen und drei Transformationslängen beantwortet sie die Teilfrage nach dem Skalierungsverhalten. Mitgeprüft wird eine Annahme des Versuchsaufbaus selbst. Dass beide Seiten für die Verstärkungsstufe praktisch identischen Code erzeugen und diese deshalb als Auflösungsgrenze taugt (Abschnitt 4.6.2). Fällt diese Annahme durch, ist die Grenze anders zu bestimmen.

Der *Optimierungs-Sweep* prüft zwei Behauptungen. Die erste lautet, dass Compileroptimierungen auf beide Implementierungen gleich wirken. Die zweite stammt aus der Literatur und besagt, dass sich vektorisierbare Ausdrücke von solchen mit Datenabhängigkeiten trennen lassen und nur die erste Gruppe von der automatischen Vektorisierung profitiert (Scaringella et al., 2003). Als Ergebnis liefert die Reihe dieselbe Laufzeitmessung unter sieben Flagsätzen. Bestätigt wäre die zweite Behauptung, wenn Verstärkungsstufe und FIR-Filter beim Übergang zu `-O3` deutlich gewinnen, Akkumulator und Biquad-Filter dagegen kaum. Widerlegt wäre die erste, wenn ein Flagsatz eine Seite bevorzugt. Bei `-ffast-math` ist genau das zu erwarten, weil das Umordnen von Gleitkommaoperationen die Abhängigkeitskette rekursiver Filter verlängern kann (Parhi, 1999). Beantwortet wird damit die Teilfrage nach dem Einfluss von Compileroptimierung und Vektorisierung.

4.7.3 Ressourcenverbrauch

Die *Binärgrößenmessung* prüft die Behauptung, dass der Faust-Compiler die Berechnung ausrollt und die manuelle Implementierung sie in Schleifen belässt. Trifft sie zu, so wächst die Codegröße der Faust-Variante mit der Filterlänge und der Transformationslänge, während die der C++-Referenz gleich bleibt. Als Ergebnis liefert die Reihe die gestrippte Größe je Baustein abzüglich einer leeren Basislinie, für drei Filterlängen und

die FFT. Beantwortet wird die Teilfrage nach der Binärgröße. Das Ergebnis ist zugleich der Beleg für den Codegenerierungsunterschied.

Die *Speichermessung* prüft die Angabe, dass Faust Code mit deterministischem und konstantem Speicherverbrauch erzeugt (Orlarey et al., 2009). Als Ergebnis liefert sie die Zustandsgröße der DSP-Klasse, den maximalen Speicherbedarf des Prozesses, die Heap-Spitze und die Zahl der Speicheranforderungen innerhalb von `compute()`. Beantwortet wird die Teilfrage nach dem Speicherverbrauch. Zustandsgröße und Heap-Spitze sind dabei gemeinsam zu lesen, weil die Faust-Variante ihren Zustand im Objekt hält, die manuelle Implementierung ihn dagegen dynamisch anfordert.

4.7.4 Entwicklungsaufwand

Der *Quelltextumfang* prüft die Behauptung, dass Faust-Beschreibungen kürzer sind als handgeschriebene C++-Implementierungen. Für physikalische Modelle wird dieser Vorteil mit 22,91 bis 80,20 Prozent angegeben (Michon & Smith, 2011). Offen ist, ob er sich auf elementare Bausteine übertragen lässt. Als Ergebnis liefert die Reihe die Codezeilen getrennt nach Faust-Beschreibung, C++-Wrapper und manueller Implementierung. Die Trennung ist entscheidend, weil der Wrapper für alle Bausteine nahezu gleich ist. Zählt man ihn mit, kann der Vorteil bei kleinen Bausteinen verschwinden. Beantwortet wird die Teilfrage nach Codelänge und Implementierungskomplexität.

Die *Übersetzungsdauer* des Faust-Compilers beantwortet keine der Teilfragen. Sie wird dennoch erhoben, weil sie das Ausrollen von einer zweiten Seite zeigt. Was sich in der Binärgröße als gewachsener Code niederschlägt, muss zuvor erzeugt werden. Als Ergebnis liefert die Reihe die Übersetzungsdauer über wachsende Transformationslängen und Rekursionstiefen, mit Abbruch nach 30 Sekunden. Für die praktische Arbeit mit der Sprache ist das ein spürbarer Faktor.

4.7.5 Einordnung und Grenzen des Messplans

Die neun Reihen verteilen sich damit auf vier Aufgaben. Die ersten drei sichern die Gültigkeit, die beiden folgenden beantworten die eigentliche Frage, drei weitere decken die Nebenaspekte Binärgröße, Speicher und Codelänge ab, und die letzte stützt die Einordnung. Zwei der geprüften Behauptungen stammen nicht aus der Literatur, sondern aus dem Versuchsaufbau selbst. Dass beide Implementierungen dasselbe berechnen, und dass die Verstärkungsstufe als Auflösungsgrenze taugt. Beide sind in Kapitel 6 ausdrücklich zu prüfen und nicht vorauszusetzen.

Drei Größen werden bewusst nicht erhoben. Es wird keine Verteilung der Rechenzeit einzelner Blöcke ausgewertet, weshalb sich zur Einhaltung der Echtzeitfrist keine Aussage treffen lässt. Es wird keine vollständige Signalkette gemessen, sondern nur einzelne Bausteine. Und es werden keine alternativen Codegenerierungsmodi von Faust untersucht. Die Gründe sind in den jeweiligen Abschnitten genannt. Kapitel 7 greift sie gesammelt auf.

4.8 Zusammenfassung

Der Versuchsaufbau vergleicht fünf DSP-Bausteine mit unterschiedlichen Datenabhängigkeiten: eine Verstärkungsstufe ohne Rückkopplung, einen Akkumulator als reinen Rekursionsfall, einen Biquad-Filter, FIR-Filter in drei Längen und FFTs in drei Größen. Beide Implementierungen verwenden dieselben Koeffizienten, dieselben Testdaten, dieselbe Schnittstelle und dieselben Compilerflags. SIMD-Intrinsics und externe DSP-Bibliotheken sind auf beiden Seiten ausgeschlossen. Gemessen werden CPU-Zeit je Block, Streuung, Binärgröße, Speicherbedarf, numerische Genauigkeit und Codelänge. Die Auswertung stützt sich auf Mediane, Bootstrap-Konfidenzintervalle und einen verteilungsfreien Test und kennzeichnet Effekte unterhalb der Auflösungsgrenze des Aufbaus als nicht auflösbar. Das folgende Kapitel beschreibt, wie diese Festlegungen im Quelltext umgesetzt sind.

Kapitel 5

Implementierung der Benchmarks

Dieses Kapitel beschreibt, wie diese Festlegungen im Quelltext umgesetzt sind. Es zeigt die Faust-Beschreibungen und die manuellen C++-Implementierungen der fünf Bausteine, die gemeinsame Schnittstelle, über die beide Varianten beschrieben werden, den Aufbau des Build-Systems sowie die Maßnahmen, die eine faire Vergleichsbasis herstellen und die Gleichwertigkeit beider Varianten nachweisen. Die Messergebnisse selbst folgen in Kapitel 6.

Abbildung 5.1 zeigt den Weg, den beide Varianten nehmen. Aus der Faust-Quellbeschreibung erzeugt der Faust-Compiler eine C++-Klasse. Ein Wrapper bindet diese Klasse an dieselbe Schnittstelle an, die auch die manuelle Implementierung erfüllt. Ab dieser Schnittstelle unterscheiden sich die beiden Varianten nicht mehr. Sie werden vom selben Messprogramm, mit denselben Testdaten und mit denselben Compilerflags verarbeitet.

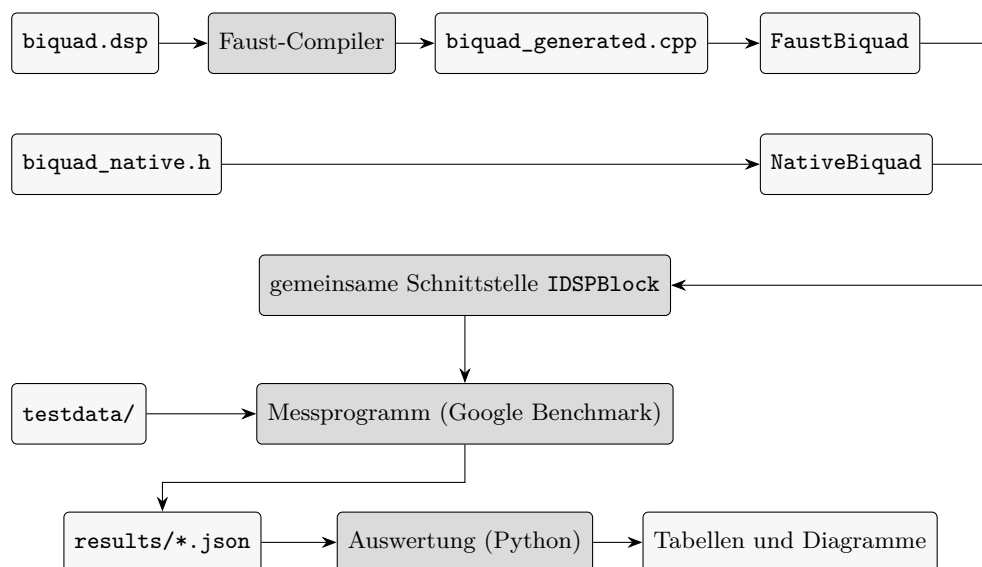


Abbildung 5.1: Vom Quelltext zum Messwert am Beispiel des Biquad-Filters. Der Faust-Compiler erzeugt eine C++-Klasse aus der .dsp-Datei, die ein Wrapper an dieselbe Schnittstelle anbindet wie die manuelle Implementierung und dann identisch verläuft.

Programm 5.1: Die fünf Faust-Quellbeschreibungen

```

1  // gain.dsp
2  process = *(hslider("gain", 0.5, 0, 2, 0.01));
3
4  // accum.dsp
5  process = + ~ _;
6
7  // biquad.dsp
8  process = fi.tf2(0.00255054, 0.00510107, 0.00255054,
9  -1.85214649, 0.86234863);
10
11 // fir32.dsp (fir64.dsp und fir256.dsp unterscheiden sich nur in N)
12 N = 32;
13 process = fi.fir(par(i, N, 2.0*(i+1)/(N*(N+1))));
14
15 // fft.dsp
16 N = 64;
17 process = an.rtocv(N) : an.fft(N);
18

```

5.1 Umsetzung des FAUST- und C++-Code

+Das Projekt trennt die beiden Varianten in eigene Verzeichnisse auf. Die Faust-Quellbeschreibungen liegen in `dsp/`, die zugehörigen Wrapper in `src/faust/` und die manuellen Implementierungen in `src/native`. Tabelle 5.1 ordnet jedem Baustein aus Tabelle 4.1 die beiden Dateien zu.

Tabelle 5.1: Zuordnung der Bausteine zu den Quelldateien

Baustein	Faust-Beschreibung	manuelle Implementierung
Gain	<code>dsp/gain.dsp</code>	<code>src/native/gain_native.h</code>
Akkumulator	<code>dsp/accum.dsp</code>	<code>src/native/accum_native.h</code>
Biquad-Filter	<code>dsp/biquad.dsp</code>	<code>src/native/biquad_native.h</code>
FIR-Filter	<code>dsp/fir32/64/256.dsp</code>	<code>src/native/fir_native.h</code>
FFT	<code>dsp/fft.dsp</code>	<code>src/native/fft_native.h</code>

Die Faust-Seite

Programm 5.1 zeigt die fünf Quellbeschreibungen vollständig. Sie sind jeweils ein bis drei Zeilen lang, weil die Signalverarbeitung nicht ausprogrammiert, sondern als Komposition benannt wird.

Die Verstärkungsstufe ist eine Multiplikation mit einem Bedienelement, dessen Vorgabewert 0,5 beträgt. Der Akkumulator besteht aus dem Rekursionsoperator aus Tabelle 3.1 in seiner kürzesten Form. Der Biquad-Filter und die FIR-Filter verwenden Bibliotheksfunktionen. `fi.tf2` nimmt die fünf Koeffizienten entgegen, `fi.fir` eine Ko-

Programm 5.2: Manuelle Biquad-Implementierung in Direktform I

```

1   for (int i = 0; i < count; ++i) {
2       float x0 = in[i];
3       float y0 = b0_*x0 + b1_*x1_ + b2_*x2_ - a1_*y1_ - a2_*y2_;
4       x2_ = x1_; x1_ = x0;
5       y2_ = y1_; y1_ = y0;
6       out[i] = y0;
7   }
8

```

effizientenliste, die `par` nach der Vorschrift aus Abschnitt 4.1 erzeugt. Die FFT entsteht aus zwei hintereinandergeschalteten Bibliotheksfunktionen: `an.rtcv` bildet aus dem Eingangssignal ein gleitendes Fenster der Länge N , `an.fft` transformiert dieses Fenster.

Die fünf Beschreibungen enthalten keine Angabe darüber, wie die Berechnung abläuft. Weder Schleifen noch Puffer noch Indexrechnung kommen vor. Genau diesen Teil ergänzt der Faust-Compiler. Sie interpretiert aus dem Rechenoperationen den optimalen C++-Code.

Die manuellen Implementierungen sind Header-only-Klassen. Für Gain und Akkumulator ist die Umsetzung unmittelbar eine Schleife über die Samples mit einer Multiplikation beziehungsweise einer Addition auf einer im Objekt gehaltenen Zustandsvariablen.

Der Biquad-Filter folgt der Direktform I (Abbildung 4.4a). Programm 5.2 zeigt die Schleife. Sie hält die beiden vorherigen Ein- und Ausgangswerte und schiebt sie nach jedem Sample weiter. Die Faust-Bibliotheksfunktion `fi.tf2` ergibt dagegen die Direktform II (Abbildung 4.4b). Die Entscheidung, jede Seite in ihrer naheliegenden Form zu formulieren, ist in Abschnitt 4.1 begründet.

Die Faust-Bibliotheksfunktion `fi.tf2` ergibt dagegen die Direktform II mit nur einer Verzögerungskette und zwei gespeicherten Werten. Abbildung 4.4 stellt beide Formen gegenüber. Der Unterschied ist keine Nachlässigkeit, sondern die in Abschnitt 4.1 begründete Entscheidung, jede Seite in der für sie naheliegenden Form zu formulieren.

Der FIR-Filter ist ein Klassen-Template über die Filterlänge. `static_assert` sichert diese Voraussetzung zur Übersetzungszeit ab. Die Koeffizienten werden im Konstruktor nach derselben Vorschrift berechnet wie in der Faust-Beschreibung. Der Faust-Compiler wählt hier einen anderen Weg und schreibt die gesamte Faltung als eine einzige Summe aus. Abbildung 5.2 zeigt beide Formen nebeneinander.

Die FFT ist der aufwendigste der fünf Bausteine. Die manuelle Implementierung ist eine iterative Radix-2-FFT nach Cooley-Tukey. Die Bit-Umkehrtabelle und die Twiddle-Faktoren werden einmalig in `initForSize()` berechnet und anschließend nur noch gelesen. In der Verarbeitungsschleife wird für jedes Eingangssample das Fenster aus dem Verlaufspuffer serialisiert, transformiert und als Real- und Imaginärteil auf $2N$ Ausgänge geschrieben.

Zwei Festlegungen sind dabei wesentlich, weil sie darüber entscheiden, ob beide Varianten überhaupt dasselbe berechnen. Erstens wird bewusst keine Fremdbibliothek verwendet. Eine frühere Fassung des Versuchsaufbaus nutzte FFTW; damit hätte der

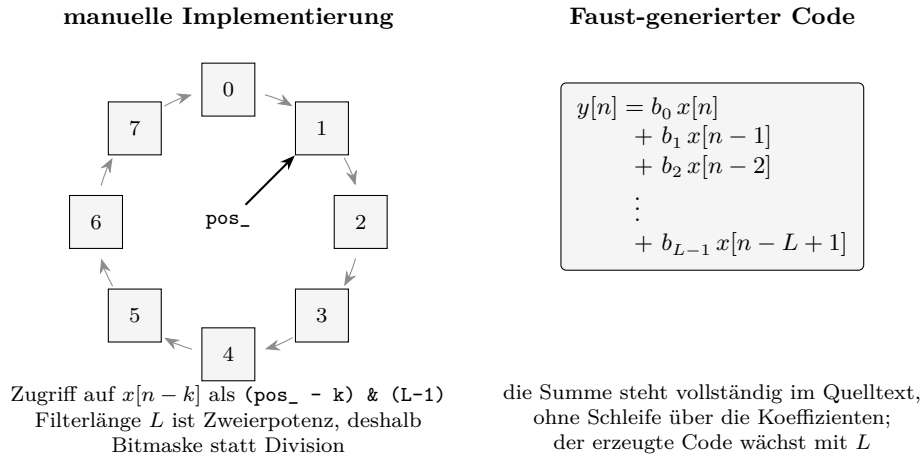


Abbildung 5.2: Zwei Wege zur selben Faltung. Die manuelle Implementierung hält die vergangenen Eingangswerte in einem Ringpuffer und läuft in einer Schleife über die Koeffizienten. Der Faust-Compiler schreibt dieselbe Summe vollständig aus.

Programm 5.3: Die gemeinsame Schnittstelle IDSPBlock

```

1  class IDSPBlock {
2      public:
3          virtual void init(int sampleRate) = 0;
4          virtual void compute(int count, float** inputs, float** outputs) = 0;
5          virtual int  getNumInputs() = 0;
6          virtual int  getNumOutputs() = 0;
7          virtual ~IDSPBlock() = default;
8  };
9

```

Vergleich Faust gegen eine handoptimierte Bibliothek mit Assembler-Kernen gestellt und nicht gegen manuell geschriebenen C++-Code. Zweitens serialisiert `an.rtcv` das Analysefenster mit dem neuesten Sample zuerst. Die manuelle Implementierung folgt derselben Reihenfolge. Wird das Fenster in der natürlichen Zeitrichtung gefüllt, berechnen beide Varianten unterschiedliche Spektren, ohne dass eine von beiden falsch wäre. Die Reihenfolge ist deshalb als Übersetzungsschalter im Quelltext festgehalten und dokumentiert.

5.2 Gemeinsame Test-Schnittstelle

Damit dieselben Messprogramme beide Varianten vermessen können, erfüllen beide dieselbe abstrakte Schnittstelle. Programm 5.3 zeigt sie vollständig.

Abbildung 5.3 zeigt die Vererbungsbeziehung und die Nutzer der Schnittstelle. Diese Aufteilung hat drei Folgen für die Messung.

1. Jedes Messprogramm ist eine Template-Funktion über den Typ des Bausteins. Faust- und C++-Variante durchlaufen deshalb denselben Quelltext.

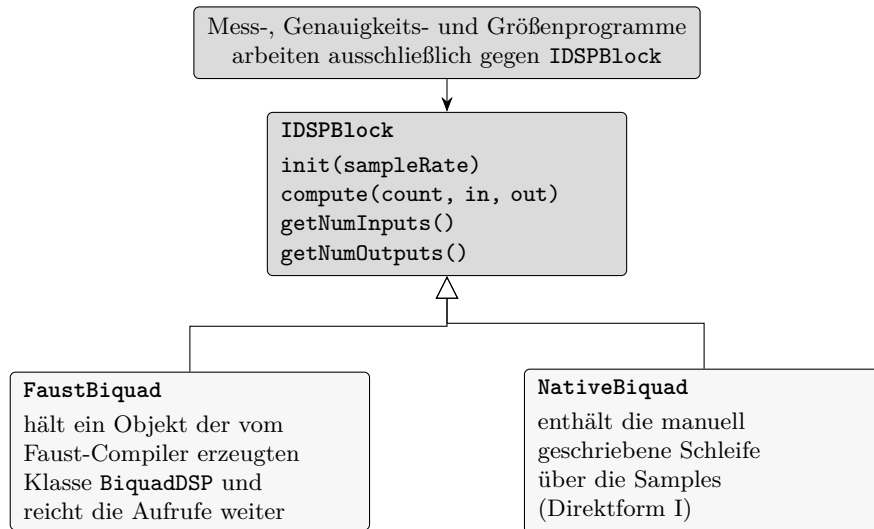


Abbildung 5.3: Gemeinsame Test-Schnittstelle. Beide Varianten erben von derselben abstrakten Klasse. Die Messprogramme kennen nur diese Schnittstelle und können deshalb für beide Varianten unverändert verwendet werden.

2. Der Wrapper ist für alle Bausteine nahezu identisch und gehört nicht zur eigentlichen Beschreibungsleistung. Die Codelängenmessung aus Abschnitt 4.2.2 zählt ihn deshalb getrennt.
3. Der Aufwand des Wrappers beschränkt sich auf einen virtuellen Aufruf je Block, nicht je Sample. Er fällt für beide Varianten gleichermaßen an und geht damit in den berichteten Verhältnissen nicht hervor.

Zwei Bausteine erfordern eine Abweichung von diesem Muster. Bei den FIR-Filtern bindet der gemeinsame Wrapper bewusst keine der generierten Dateien ein. Erst die drei größenspezifischen Wrapper tun das. Ohne diese Trennung enthielte jede Übersetzungseinheit alle drei generierten Klassen, und die Binärgrößenmessung wäre nicht gültig. Bei der FFT ist der Unterschied woanders. Die Faust-Klasse ist je Transformationslänge eine eigene, vollständig ausgerollte Klasse. Wobei die manuelle Implementierung die Länge erst zur Laufzeit in `initForSize()` erhält. Deshalb wird je Länge ein eigenes ausführbares Programm erzeugt.

5.3 Build-System

Das Projekt wird mit CMake ab Version 3.20 und Ninja übersetzt, als Sprachstandard dient C++17. Google Benchmark ist als Submodul eingebunden und wird gemeinsam mit dem Projekt gebaut, sodass die Messbibliothek dieselben Flags erhält wie der Messcode.

Der Aufruf des Faust-Compilers ist als CMake-Funktion gekapselt (Programm 5.4). Sie meldet die erzeugte Datei als Build-Artefakt an, sodass CMake die Abhängigkeit zur Quellbeschreibung kennt und nach einer Änderung an der `.dsp`-Datei neu übersetzt.

Die Option `-cn` legt den Namen der erzeugten Klasse fest. Weitere Schalter werden

Programm 5.4: Einbindung des Faust-Compilers in das Build-System

```
1 function(faust_compile DSP_FILE OUT_CPP CLASS_NAME)
2   add_custom_command(
3     OUTPUT ${OUT_CPP}
4     COMMAND ${FAUST_EXECUTABLE} -cn ${CLASS_NAME}
5     ${DSP_FILE} -o ${OUT_CPP}
6     DEPENDS ${DSP_FILE}
7     VERBATIM)
8   endfunction()
9
```

nicht gesetzt; es gilt damit das voreingestellte C++-Backend im skalaren Modus, wie in Abschnitt 4.4 begründet.

Die generierten Dateien werden nicht getrennt übersetzt, sondern vom jeweiligen Wrapper eingebunden. Die Faust-Bausteine sind deshalb reine Schnittstellen-Bibliotheken, die nur Include-Pfade beisteuern. Der Vorteil dieser Konstruktion zeigt sich bei den größenabhängigen Bausteinen: Für jede FFT-Länge erzeugt das Build-System aus einer Vorlage eine eigene Quellbeschreibung in einem eigenen Verzeichnis und übersetzt sie zu einer eigenen Klasse. Der Wrapper bleibt dabei unverändert und sieht über den Include-Pfad genau die für sein Ziel erzeugte Datei.

Die Optimierungsflags stehen in einer einzigen Cache-Variablen, die die Release-Flags überschreibt. Zusätzlich schreibt der Build eine Datei mit den tatsächlich verwendeten Übersetzungsbefehlen, die Abschnitt 5.4 für die Prüfung der Vergleichsbasis heranzieht. Jede Konfiguration der Flag-Matrix aus Abschnitt 4.4 erhält ein eigenes Build-Verzeichnis.

Neben den Messprogrammen entstehen die Probe-Programme für die Binärgrößen- und Speichermessung. Sie stammen alle aus einer einzigen Quelldatei; Header und Klassenname des zu messenden Bausteins werden ihr über Präprozessor-Definitionen mitgegeben. Dadurch unterscheiden sich die Probe-Programme untereinander ausschließlich im gemessenen Baustein.

5.4 Sicherstellung einer fairen Vergleichsbasis

Der Vergleich ist nur dann aussagekräftig, wenn sich die beiden Varianten ausschließlich in dem unterscheiden, was untersucht werden soll, nämlich in der Art, wie der C++-Code entstanden ist. Der Aufbau stellt das auf fünf Wegen sicher.

Gleiche Compilerflags. Beide Varianten werden im selben Build-Verzeichnis mit derselben Flag-Variablen übersetzt. Die Absicht allein genügt allerdings nicht als Nachweis. Ein eigenes Prüfskript vergleicht deshalb in drei Stufen: den Build-Typ im Cache, die gesetzten Release-Flags und schließlich die tatsächlichen Übersetzungsbefehle jeder einzelnen Übersetzungseinheit. Erst die dritte Stufe ist ein Beweis; die ersten beiden zeigen nur die Absicht. Weicht eine Übersetzungseinheit ab, bricht die Messkette ab.

Gleiche Eingangsdaten. Beide Varianten lesen dieselbe Datei mit denselben Werten. Die Testsignale entstehen einmalig aus dem Skript aus Abschnitt 4.5 und liegen als binäre Rohdaten vor. Ein Zufallsgenerator mit festem Startwert stellt sicher, dass wiederholte Läufe dieselben Signale verwenden.

Gleicher Messrahmen. Faust- und C++-Variante werden im selben Prozess, im selben Zeitfenster und über dieselbe Template-Funktion gemessen. Innerhalb der Messschleife verhindern für beide dieselben Hilfsfunktionen, dass der Compiler den Aufruf als toten Code entfernt.

Gleiche Mittel. Auf beiden Seiten sind explizit programmierte SIMD-Intrinsics und externe DSP-Bibliotheken ausgeschlossen. Die automatische Vektorisierung des C++-Compilers bleibt für beide aktiv.

Getrennte Programme je Messgröße. Die Genauigkeitsprogramme werden ausdrücklich ohne gelockerte Gleitkommaregeln übersetzt, weil dort die tatsächliche Rundung in einfacher Genauigkeit gemessen werden soll und nicht die Wirkung einer Compileroptimierung. Für die Binärgrößenmessung enthält jedes Probe-Programm genau einen Baustein, damit die Differenz zur Basislinie diesem Baustein zugeordnet werden kann.

Zwei Unterschiede bleiben bestehen und lassen sich mit diesem Aufbau nicht auflösen. Der Biquad-Filter liegt auf beiden Seiten in unterschiedlicher Direktform vor, und die FFT ist auf der Faust-Seite je Länge ausgerollt, während sie auf der C++-Seite zur Laufzeit parametrisiert wird. Beide Unterschiede folgen daraus, dass jede Seite in ihrer naheliegenden Form formuliert wurde. Sie werden hier benannt und in Kapitel 6 bei der Deutung der betroffenen Messwerte erneut aufgegriffen.

5.5 Verifikation der Ergebnisse

Ein Laufzeitvergleich zweier Implementierungen ist wertlos, solange nicht gezeigt ist, dass beide dasselbe berechnen. Die Verifikation läuft deshalb vor jeder Laufzeitmessung und in zwei Stufen.

Die erste Stufe vergleicht die Faust-Variante unmittelbar mit der C++-Variante. Beide verarbeiten nacheinander dieselben Testsignale aus Abschnitt 4.5 in allen vorgesehenen Längen. Ausgewertet werden der maximale Absolutfehler, das quadratische Mittel des Fehlers und der größte relative Fehler zwischen den beiden Ausgangssignalen.

Diese Stufe allein genügt jedoch nicht. Stimmen beide Varianten überein, so ist damit nur gezeigt, dass sie sich einig sind, nicht, dass sie recht haben. Ein systematischer Fehler, den beide gemeinsam machen, bliebe unentdeckt. Ein solcher Fall ist im Verlauf der Arbeit tatsächlich aufgetreten: Beim Vergleich der FFT ergab eine der beiden möglichen Fensterorientierungen ein exakt negiertes Spektrum. Ohne eine dritte, unabhängige Instanz wäre nicht entscheidbar gewesen, welche der beiden Varianten der Konvention folgt.

Die zweite Stufe führt deshalb eine Referenz ein. Für jeden Baustein und jedes Testsignal berechnet ein NumPy-Skript die Ausgabe in doppelter Genauigkeit und legt sie als Rohdatei ab. Die Referenzalgorithmen bilden die Struktur der C++-Bausteine nach

und unterscheiden sich von ihnen nur im Datentyp. Beide Implementierungen werden anschließend gegen diese Referenz gemessen. Berichtet werden der maximale und der mittlere Absolutfehler sowie das quadratische Mittel des Fehlers. Der relative Fehler wird auf den größten Betrag der Referenz bezogen und als Vielfaches der Maschinengenauigkeit von `float` ausgedrückt.

Zu jedem Baustein gehört eine Toleranzschwelle, die dem Prüfprogramm als Argument übergeben wird. Überschreitet ein Signal diese Schwelle, liefert das Programm einen Fehlercode zurück und die Messkette bricht ab. Laufzeitzahlen zweier Implementierungen, die nicht dasselbe berechnen, kämen damit gar nicht erst zustande.

Zwei weitere Prüfungen ergänzen den Nachweis. Ein eigenes Programm lässt den Akkumulator über viele Millionen Blöcke laufen und protokolliert seinen Zustand. Damit lässt sich beobachten, wie der Wert über die Zeit anwächst und an welchem Punkt die Auflösung der einfachen Genauigkeit erschöpft ist. Diese Prüfung ist kein Genauigkeitsnachweis, sondern belegt, dass der beobachtete Fehler des Akkumulators eine Eigenschaft des Bausteins ist und keine Eigenschaft einer der beiden Implementierungen. Die Probe-Programme zählen darüber hinaus über einen überladenen Speicheroperator die Speicheranforderungen innerhalb der Verarbeitungsmethode mit. Diese Zahl muss für beide Varianten null sein, weil eine Speicheranforderung im Audio-Pfad die Echtzeitfähigkeit gefährdet.

Damit sind die Vorkehrungen beschrieben, unter denen gemessen wird. Kapitel 6 berichtet die Ergebnisse dieser Messungen.

Kapitel 6

Evaluation der Messergebnisse

6.1 Mikrobenchmark-Ergebnisse

je Baustein: CPU-Zeit/Block; Durchsatz; Speicherverbrauch, Binaergröße Grafiken

6.2 Einfluss von Compileroptimierungen und Codegenerierung

-O0 -> -O3 und -fastmath

Vektorisierung wurde nicht berücksichtigt; Gibt schon eine studie

6.3 Numerische Genauigkeit

6.4 Codelänge und Implementierungskomplexität

6.5 Zusammenfassung der Messergebnisse

Kapitel 7

Diskussion

- 7.1 Interpretation der Ergebnisse im Hinblick auf die Forschungsfrage
- 7.2 Limitationen des Versuchsaufbaus
- 7.3 Bedrohung der Validität

Kapitel 8

Zusammenfassung und Ausblick

8.1 Zusammenfassung der zentralen Ergebnisse

8.2 Ausblick auf künftige Forschungsarbeiten

Quellenverzeichnis

Literatur

- Buttazzo, G. C. (1997). *Hard real-time computing systems: predictable scheduling algorithms and applications*. Springer. (Siehe S. 28).
- Cooley, J. W., & Tukey, J. W. (1965). An Algorithm for the Machine Calculation of Complex Fourier Series. *Mathematics of Computation*, 19(90), 297–301. <https://doi.org/10.1090/S0025-5718-1965-0178586-1> (siehe S. 25).
- Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall. (Siehe S. 31).
- Google LLC. (2023). *Google Benchmark: A microbenchmark support library* [Library for microbenchmarking C++ code]. <https://github.com/google/benchmark> (siehe S. 26).
- Ifeachor, E. C., & Jervis, B. W. (2002). *Digital signal processing: a practical approach*. Pearson Education. (Siehe S. 1).
- Letz, S., Orlarey, Y., & Foer, D. (2010). Work Stealing Scheduler for Automatic Parallelization in Faust. In LAC (Hrsg.), *Proceedings of Linux Audio Conference*. <https://hal.science/hal-02158924> (siehe S. 2, 17, 29).
- Levine, N., & Skolnick, D. (1997). DSP 101 Part 3: Implement Algorithms on a Hardware Platform. *Analog Dialogue*, 31(3). Verfügbar 4. August 2026 unter <https://www.analog.com/en/resources/analog-dialogue/articles/dsp-101-part-3.html> (siehe S. 1).
- Mann, H. B., & Whitney, D. R. (1947). On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other. *The Annals of Mathematical Statistics*, 18(1), 50–60. <https://doi.org/10.1214/aoms/1177730491> (siehe S. 31).
- Michon, R., & Smith, J. O. (2011). FAUST-STK: A Set of Linear and Nonlinear Physical Models for the FAUST Programming Language. *Proceedings of the 14th International Conference on Digital Audio Effects (DAFx-11)* (siehe S. 1, 2, 12, 34).
- Mytkowicz, T., Diwan, A., Hauswirth, M., & Sweeney, P. F. (2009). Producing wrong data without doing anything obviously wrong! *ACM Sigplan Notices*, 44(3), 265–276 (siehe S. 27).
- Orlarey, Y., Foer, D., & Letz, S. (2009). FAUST : an Efficient Functional Approach to DSP Programming. In E. D. FRANCE (Hrsg.), *NEW COMPUTATIONAL PARADIGMS FOR COMPUTER MUSIC* (S. 65–96). <https://hal.science/hal-02159014> (siehe S. 1, 2, 7, 10, 12–18, 34).

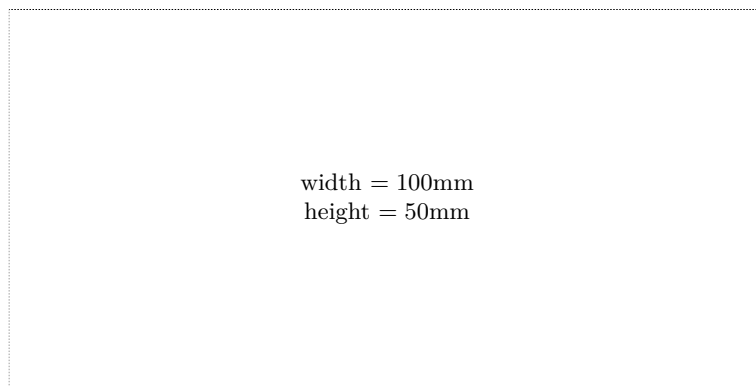
- Parhi, K. K. (1999). *VLSI Digital Signal Processing Systems: Design and Implementation*. John Wiley & Sons. (Siehe S. 29, 33).
- Rossing, T. (2007). *Springer Handbook of Acoustics*. Springer New York. https://books.google.at/books?id=4ktVwGe_dSMC (siehe S. 4).
- Scaringella, N., Orlarey, Y., Letz, S., & Fober, D. (2003). Automatic vectorization in Faust. In JIM (Hrsg.), *Actes des Journées d'Informatique Musicale JIM2003*. <https://hal.science/hal-02158949> (siehe S. 2, 7, 8, 17, 19, 29, 33).
- Smith, J. O. (2007). *Introduction to digital filters: with audio applications* (Bd. 2). Julius Smith. (Siehe S. 6–8).
- Tan, L., & Jiang, J. (2018). *Digital Signal Processing: Fundamentals and Applications*. Academic Press. <https://books.google.at/books?id=MxlxDwAAQBAJ> (siehe S. 4–6, 9, 10).
- Using the GNU Compiler Collection (GCC), Version 13.3.0* [Abschnitt „Options That Control Optimization“]. (2024). Free Software Foundation. <https://gcc.gnu.org/onlinedocs/gcc-13.3.0/gcc/Optimize-Options.html> (siehe S. 29).
- Zölzer, U. (2022). *Digital audio signal processing*. John Wiley & Sons. (Siehe S. 4, 6, 7).

Online-Quellen

- GRAME - Centre National de Création Musicale. (2025). *Faust Manual*. Verfügbar 20. Oktober 2025 unter <https://faustdoc.grame.fr/manual/introduction/> (siehe S. 1, 17).

Check Final Print Size

— Check final print size! —



— Remove this page after printing! —